M.TECH · CS-6315

Advanced Software Engineering — Visual Study Book

Reuse → Architecture → Evolution & Reengineering → Dependability, Quality & CASE → Usability & modern paradigms.
Every topic with a diagram, a real-life use case and an exam tip.

Unit I · Software Reuse
Unit II · Reuse Architecture
Unit III · Evolution & Reengineering
Unit IV · Dependability, Quality, CASE
Unit V · Usability & Paradigms

54 original diagrams · definitions to open every answer · Hinglish memory hooks · revision checklist

Table of Contents

How to use this book 3

Introduction to Software Reuse 5

1.1 What is Software Reuse? 5
1.2 Reuse Types 6
1.3 Reuse Approaches 7
1.4 Reuse Technology 8
1.5 Reuse Benefits & Barriers 8
1.6 Reuse Success & Failure Factors 9
1.7 CBSE Process 10
1.8 Reuse-Driven Software Engineering as a Business 11

Architectural Concepts in Reuse 12

2.1 Application and Component Systems 12
2.2 Application Families Allow Significant Reuse 13
2.3 Developing Application Systems from Reusable Components 13
2.4 Reuse Variability 14
2.5 Facades for Component System Internals and Externals 15
2.6 Organizing a System in Layered Architecture 15

Software Change, Evolution, Maintenance & Reengineering 17

3.1 Software Change & Software Evolution 17
3.2 Software Maintenance: Types, Models and Metrics 18
3.3 Reengineering: Process and Activities 20
3.4 Program Comprehension 21
3.5 Reverse Engineering 21
3.6 Restructuring 22
3.7 Forward Engineering 23
3.8 Re-documentation 24
3.9 Software Aging 25

Dependability, Reliability, Quality & CASE 27

4.1 Software Dependability 27
4.2 Software Safety 28
4.3 Software Availability 29
4.4 Software Reliability: Metrics, Approaches and Models 30
4.5 Software Quality & Quality Factors 32
4.6 Verification & Validation (V&V) 33
4.7 Software Quality Assurance (SQA) 34
4.8 CASE: Scope and Technology 35
4.9 CASE Support in SDLC 36
4.10 Second Generation CASE Tools 37
4.11 Architecture of a CASE Environment 37

Usability Engineering & Advanced Paradigms 39

5.1 Usability Engineering & HCI 39
5.2 Types of UI 40
5.3 Component-Based GUI Development 41
5.4 Usability Engineering Process and Methods 42
5.5 Aspect-Oriented Software Engineering (AOSE) 43
5.6 Cleanroom Software Engineering 44
5.7 Crowdsourcing in Software Engineering 45
5.8 Artificial Intelligence & Machine Learning in SDLC 45

Quick comparison sheet 48

Exam-day revision checklist 50

Recommended resources 51

How to use this book

The 3-step exam answer formula

⭐ **Exam tip — Write every answer in this order**

1. **Definition** — one line (the blue 📘 boxes give it ready-made).
2. **Explain in 3–5 lines** — plain English with the **bold** keywords.
3. **Diagram, table or example** — draw the figure from the book; neat labelled diagrams earn marks.

Box guide

Box	What it gives you
📘 Definition	Opening line for the answer
🏭 Real-life use case	Where it is used — good for "applications" and viva
💡 Why it matters	The reason a concept exists
⭐ Exam tip	Marks-scoring points and exact statements to memorise
🧠 Yaad rakho	Short Hinglish memory hook
⚠️ Careful	Common mistakes and traps
❓ Likely exam questions	Past-paper style questions for the unit

Course outcomes

CO	Description
CO1	Implement reusability mechanisms for producing application systems
CO2	Explore maintenance and reengineering approaches for legacy systems
CO3	Understand properties and methods of designing a reliable and safe system
CO4	Analyse and apply CASE in SDLC phases
CO5	Study advanced concepts across domains to produce an effective software system

Syllabus map

Unit	Focus	Main figures to practise
I	Software reuse — types, approaches, benefits, CBSE, reuse as a business	Reuse types · CBSE process · maturity ladder
II	Application vs component systems, families, variability, facades, layers	App family · facade · layered architecture
III	Change, evolution, maintenance, reengineering, aging	Lehman · reengineering pipeline · horseshoe
IV	Dependability, safety, availability, reliability, quality, V&V, SQA, CASE	Availability · Jelinski-Moranda · V-model · CASE architecture
V	HCI, usability, AOSE, Cleanroom, crowdsourcing, AI in SDLC	UI evolution · AOP terms · Cleanroom

The big picture — how the five units fit

Stage	Unit	What happens
1. Build smarter	I–II — Reuse	Assemble systems from qualified components and product-line architectures with facades and layers
2. Keep it alive	III — Evolution	Systems must change; maintain, reengineer and fight aging
3. Make it trustworthy	IV — Dependability & quality	Reliable, safe, available; V&V and SQA; CASE tools automate the lifecycle
4. Make it usable & modern	V — Usability & paradigms	Usable UIs, aspects, Cleanroom, crowds and AI help produce better software

🏭 **Real-life use case — One product, end to end**

A company builds a family of apps from shared components (Units I–II), keeps them healthy with maintenance and a reengineering of the old module (III), proves reliability and quality with V&V and CASE tooling (IV), and polishes the UI with usability testing while AI assistants speed up coding and testing (V).

GFG links in this book open a GeeksforGeeks **search** for that topic (so they never break) — click the top result. Practice: geeksforgeeks.org/software-engineering

UNIT I

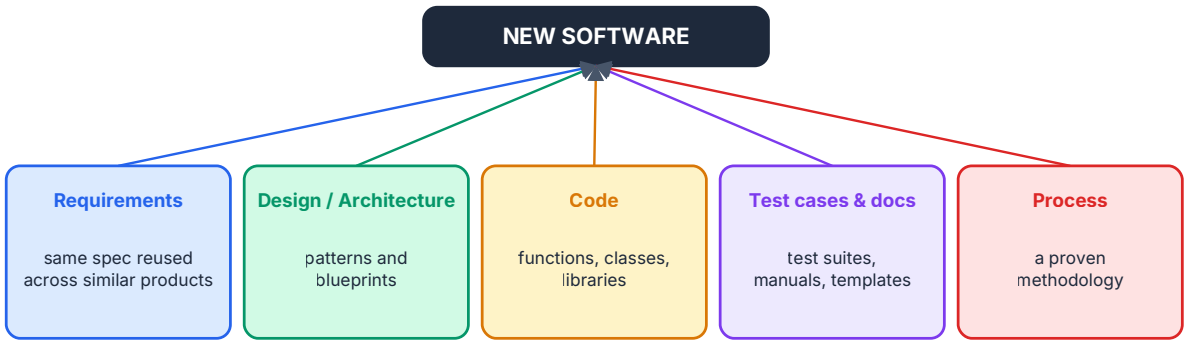
Introduction to Software Reuse

What is reuse | Reuse types | Reuse approaches | Reuse technology | Benefits & barriers | Success & failure factors | CBSE process | Reuse as a business

1.1 What is Software Reuse?

📘 **Definition**

Software reuse is building new software from **existing artifacts** — code, designs, requirements, test cases, architectures or documentation — instead of developing everything from scratch.



Reuse existing, tested artifacts instead of building from scratch

Fig. 1 — Four levels at which an artifact can be reused

Motivation: writing software is **expensive and error-prone**; something already built, tested and proven saves time, cost and risk.

Level	What is reused
Code-level	Functions, classes, libraries
Design-level	Architectural patterns, design blueprints
Requirements-level	Requirement specifications across similar products
Process-level	An entire proven development process / methodology

🏭 **Real-life use case — Real-life use**

A startup builds its login with an existing auth library, payments with a payment SDK and UI with a component kit — only the business-specific part is new code.

> 🧠 **Yaad rakho**
>
> Reuse = **"bana-banaya, test-kiya hua"** use karo. Code hi nahi — design, requirements aur process bhi reuse hote hain.

📖 GFG: Reuse Oriented Model · DRY Principle

1.2 Reuse Types

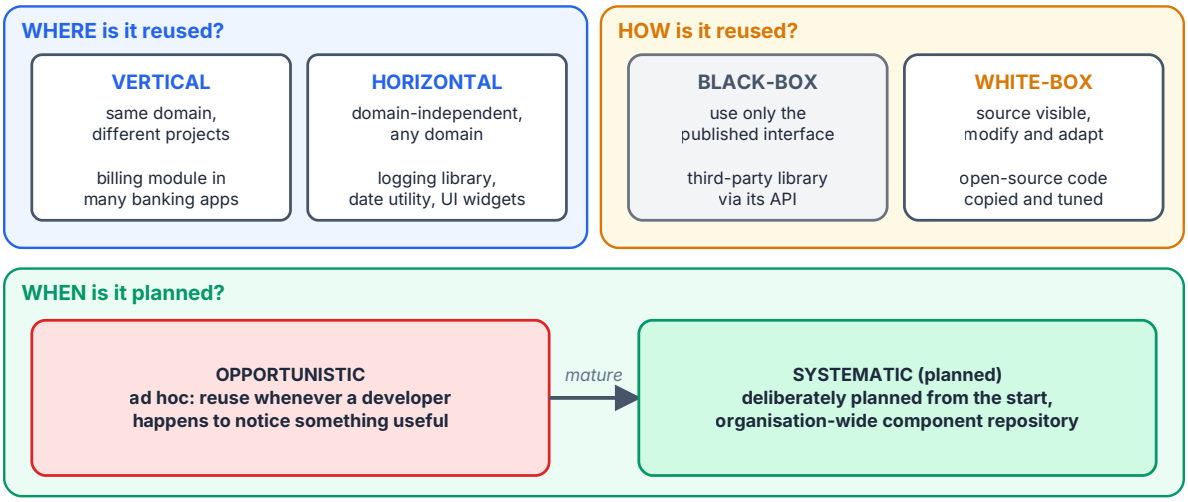


Fig. 2 — Where, how and when reuse happens — vertical/horizontal, black/white-box, opportunistic/systematic

#	Type	Meaning & example
1	**Vertical**	Reuse across projects **within the same domain** — a billing module reused by several banking apps
2	**Horizontal**	Generic, **domain-independent** components across domains — logging library, date utility, UI widgets
3	**Black-box**	Use a component only through its **published interface**, without knowing or changing internals — third-party API
4	**White-box**	Internal source is **visible and can be modified** to fit the new system
5	**Opportunistic vs systematic**	Ad hoc, whenever a developer notices something reusable *vs* deliberately planned from the start (organisation-wide component repository)

> ⭐ **Exam tip**
>
> List all 5 pairs with one example each. Vertical = **same domain**; horizontal = **across domains**.

📖 GFG: Reuse Maturity Model

1.3 Reuse Approaches

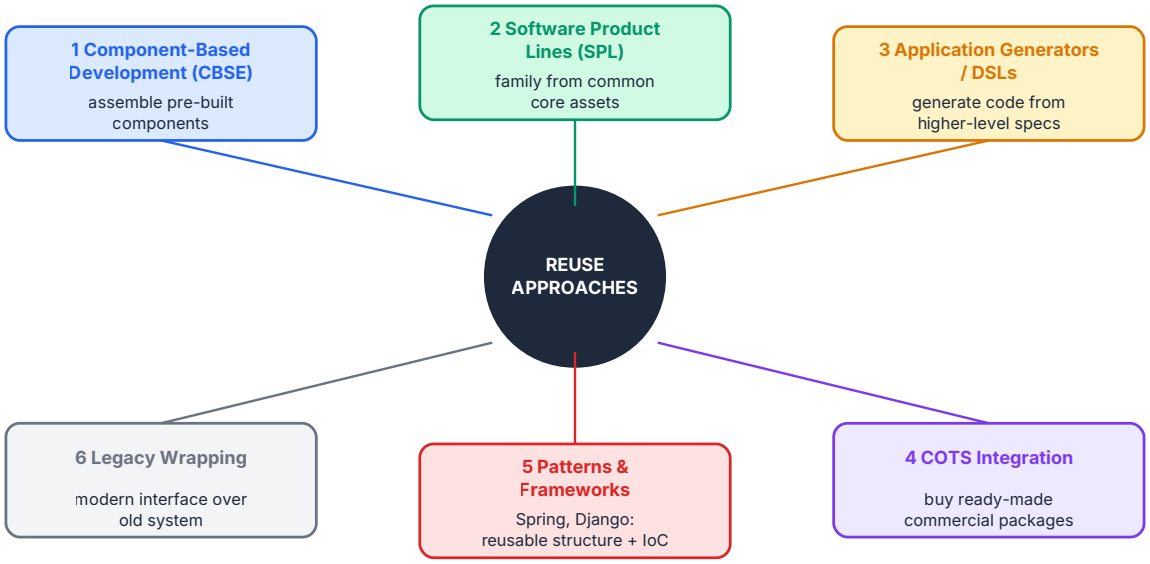


Fig. 3 — Six common approaches an organisation can use to achieve reuse

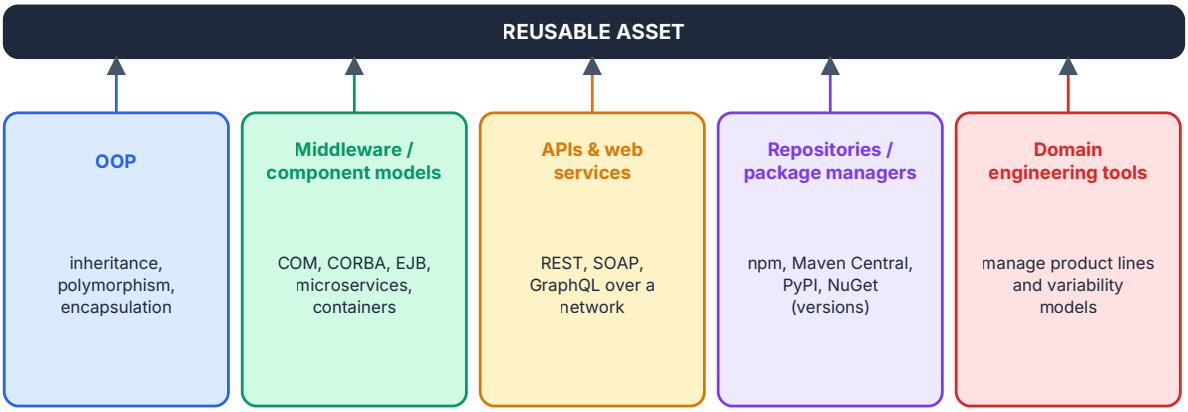
1. **Component-Based Development (CBD / CBSE)** — assemble pre-built, independently deployable components (§1.7).
2. **Software Product Lines (SPL)** — a family of related products from common core assets, tailored via variability points (Unit II).
3. **Application generators / generative programming** — templates, code generators or DSLs produce code from higher-level specs.
4. **COTS integration** — reuse ready-made commercial packages instead of building.
5. **Design patterns & frameworks** — reuse proven design solutions or whole skeletons (Spring, Django) with inversion of control.
6. **Legacy system wrapping** — put a modern interface in front of an old system so it can be reused without rewriting.

🏭 Real-life use case — Real-life use

A bank exposes its 30-year-old COBOL core through a REST wrapper so a new mobile app can reuse it — **legacy wrapping**.

📖 GFG: Component Based Software Engineering

1.4 Reuse Technology



Technologies that make reuse practically possible

Fig. 4 — Technologies that make reuse practical

Technology	How it enables reuse
OOP	Inheritance, polymorphism, encapsulation reuse classes and hierarchies
Middleware & component models	COM, CORBA, EJB (historically); microservices, containers, package managers let independent components interoperate
APIs & web services	REST, SOAP, GraphQL let one system's functionality be reused over a network
Repositories / package managers	npm, Maven Central, PyPI, NuGet — versioning and dependency management
Domain engineering tools	Build and manage product lines and their variability models

1.5 Reuse Benefits & Barriers

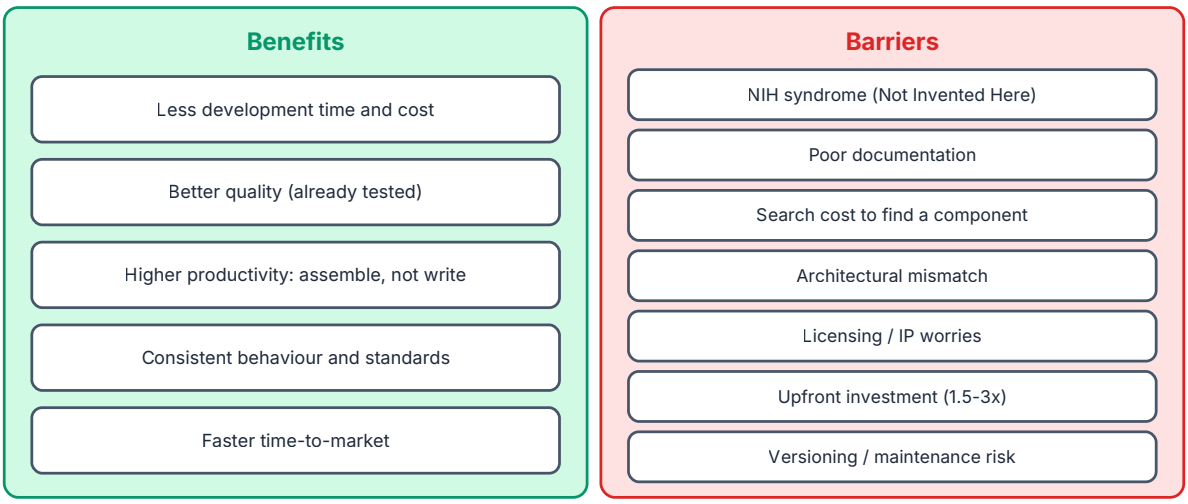


Fig. 5 — Why teams reuse, and what stops them

Benefits	Barriers
Reduced development time and cost	**NIH (Not-Invented-Here)** syndrome
Improved quality — reused parts are already tested	Poor documentation — components are hard to find and trust
Increased productivity — assemble, don't write	Discovery / search cost can exceed writing new code
Consistent behaviour and standards	Integration / interface mismatch (architectural mismatch)
Faster time-to-market	Licensing and IP concerns
	Upfront investment for generalised components
	Maintenance / versioning risk of external dependencies

> ⭐ **Exam tip**
>
> Give **5 benefits and 7 barriers**; always name **NIH syndrome** and **architectural mismatch** — they are the standard keywords.

📖 GFG: Software Reuse Benefits & Barriers

1.6 Reuse Success & Failure Factors

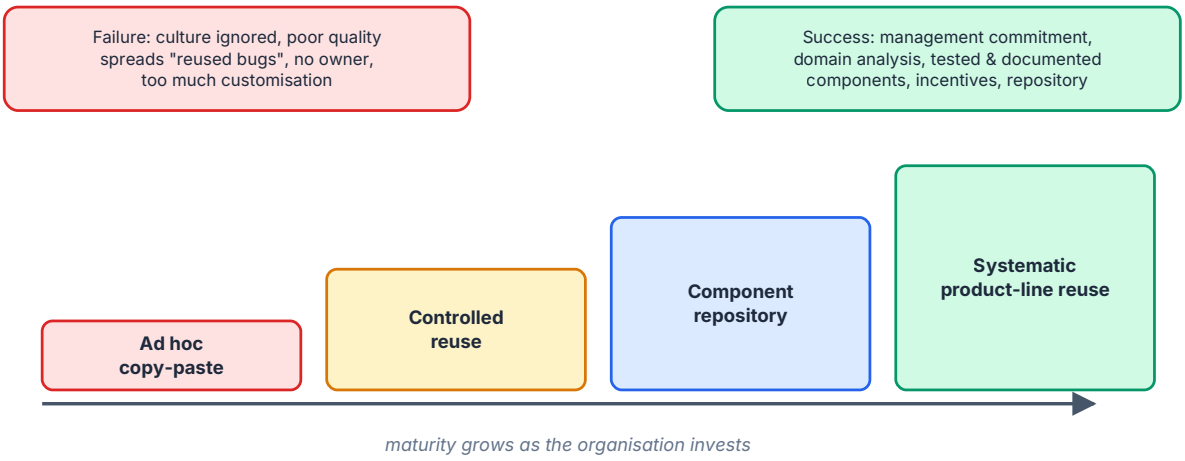


Fig. 6 — Reuse maturity ladder: ad hoc → controlled → repository → systematic product-line reuse

Success factors

1. **Management commitment** — reuse needs upfront investment and buy-in.
2. **Domain analysis** — know what is genuinely reusable across products.
3. **Well-documented, well-tested components.**
4. **Incentives** — reward creating and using reusable assets, not only shipping features.
5. **Supportive infrastructure** — repositories, search, certification processes.
6. **Gradual, staged maturity** — copy-paste → controlled reuse → systematic product lines.

Why reuse programs fail

- Treated as only a technical problem; **cultural resistance** ignored.
- Poor quality / untested components — "reused bugs" spread to every system.

- **No clear ownership** or maintenance responsibility for shared components.
- Too much customisation — components become too specific to reuse.

> 🧠 **Yaad rakho**
>
> Reuse ek **raat mein** nahi hota — maturity **step by step** badhao.

1.7 CBSE Process

> 📘 **Definition**
>
> **Component-Based Software Engineering (CBSE)** designs and builds systems from **reusable components** — independent, replaceable units with well-defined interfaces.

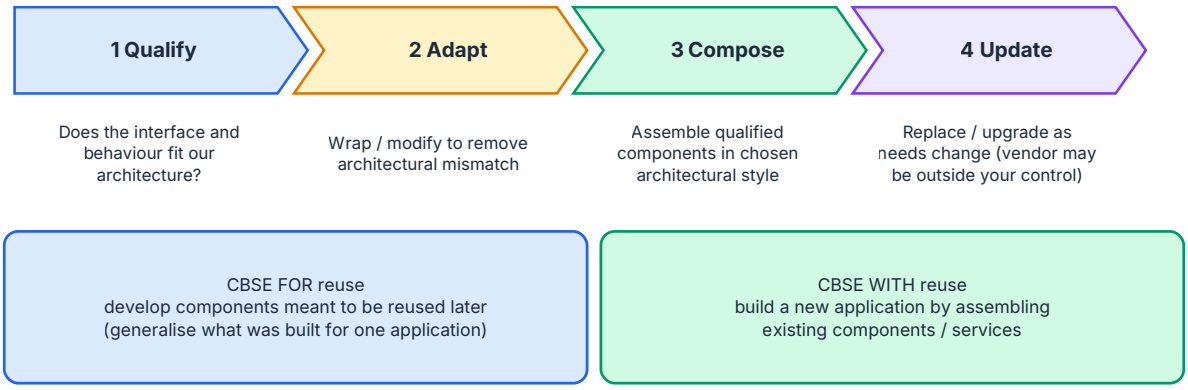


Fig. 7 — The four CBSE activities, and CBSE *for* reuse vs CBSE *with* reuse

1. **Component qualification** — does its interface and behaviour fit the target architecture?
2. **Component adaptation** — modify or wrap it to remove architectural mismatch (Unit II facades).
3. **Component composition / assembly** — integrate qualified components using an architectural style.
4. **Component update** — replace or upgrade as requirements evolve (hard when the vendor is external).

CBSE *for* reuse	CBSE *with* reuse
Develop components **meant to be reused later**, usually by generalising components built for one application	Build a **new application by assembling** existing components / services

> 🧠 **Yaad rakho**
>
> **Q-A-C-U**: Qualify, Adapt, Compose, Update. **For** = banana, **With** = lagana.

📖 GFG: CBSE

1.8 Reuse-Driven Software Engineering as a Business

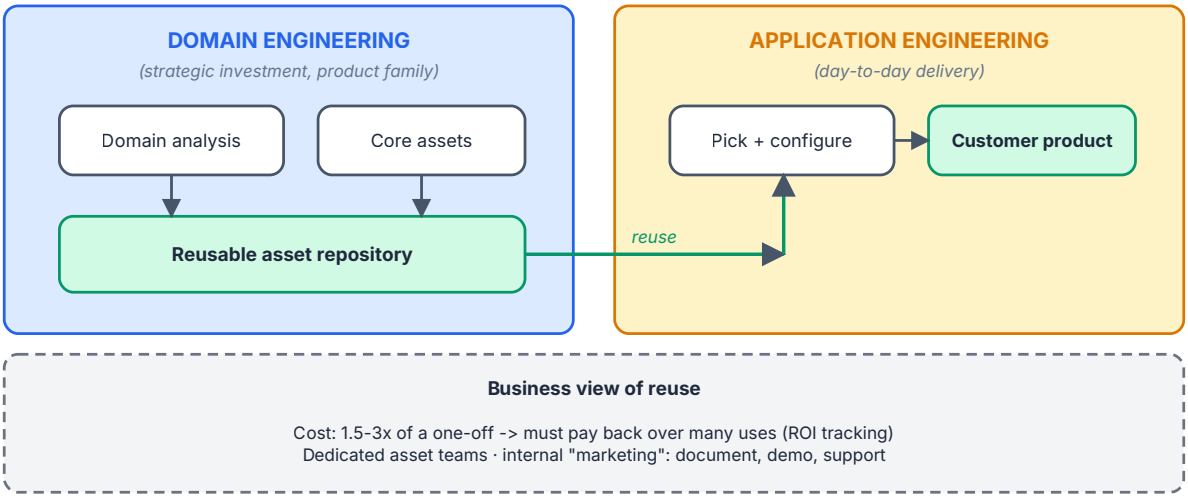


Fig. 8 — Domain engineering builds the assets; application engineering delivers products from them

Idea	Meaning
Cost / benefit	Reusable assets cost more upfront (often estimated at **1.5–3×** a one-off), so they must pay off over several uses
ROI mindset	Track how often and how effectively assets are used to justify the investment
Domain engineering	Build generic reusable assets for the whole product family — strategic investment
Application engineering	Build specific products by reusing domain assets — day-to-day delivery
Organisation structure	Separate asset / component teams from product teams so assets get long-term maintenance
Internal marketing	Document, demo and support components so other teams adopt them

🏭 **Real-life use case — Real-life use**

A company with a dedicated "platform team" that owns the shared UI kit and auth service, and product teams that consume them — domain vs application engineering in practice.

❓ **Likely exam questions — Unit I**

- Define software reuse. Levels of reuse.
- Vertical vs horizontal; black-box vs white-box reuse.
- Opportunistic vs systematic reuse.
- Explain reuse approaches (CBD, SPL, generators, COTS, patterns, wrapping).
- Technologies that support reuse.
- Benefits and barriers of reuse; what is NIH syndrome?
- Success and failure factors of a reuse programme.
- Explain the CBSE process. CBSE for vs with reuse.
- Domain engineering vs application engineering.
- Why is reuse a business decision? (1.5–3× cost, ROI)

UNIT II

Architectural Concepts in Reuse

Application vs component system | Application families | Building from components | Variability | Facades | Layered architecture

2.1 Application and Component Systems

Definition

An **application system** is the complete, end-user-facing product built for specific business requirements. A **component system** is a smaller, self-contained, reusable unit with a well-defined interface, designed to be assembled into one or more application systems.

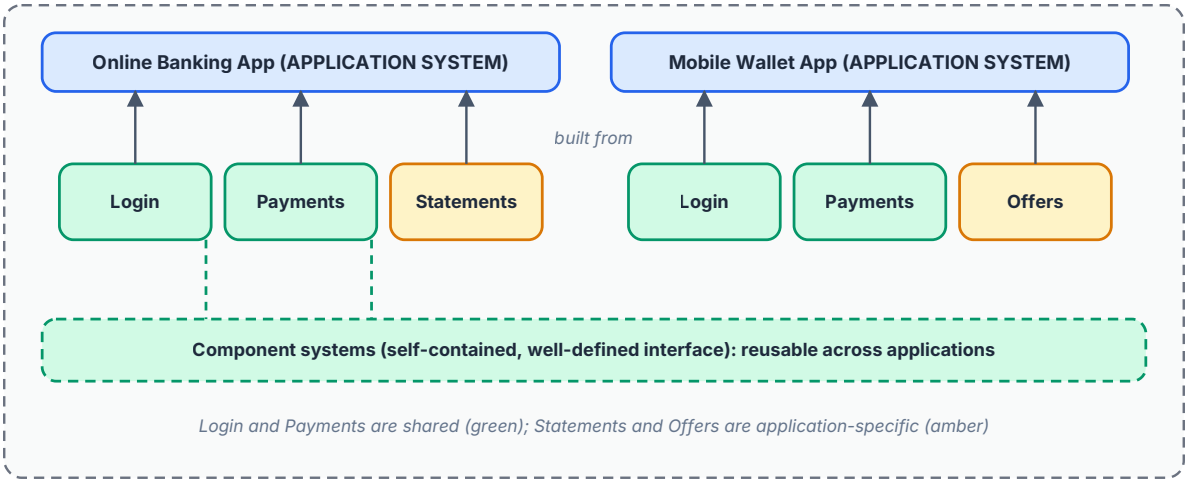


Fig. 9 — Two applications built from shared component systems

Application systems are **composed of** component systems, but a well-designed component system is deliberately **generic** enough to be reused across several applications, not tied to just one.

Real-life use case — Real-life use

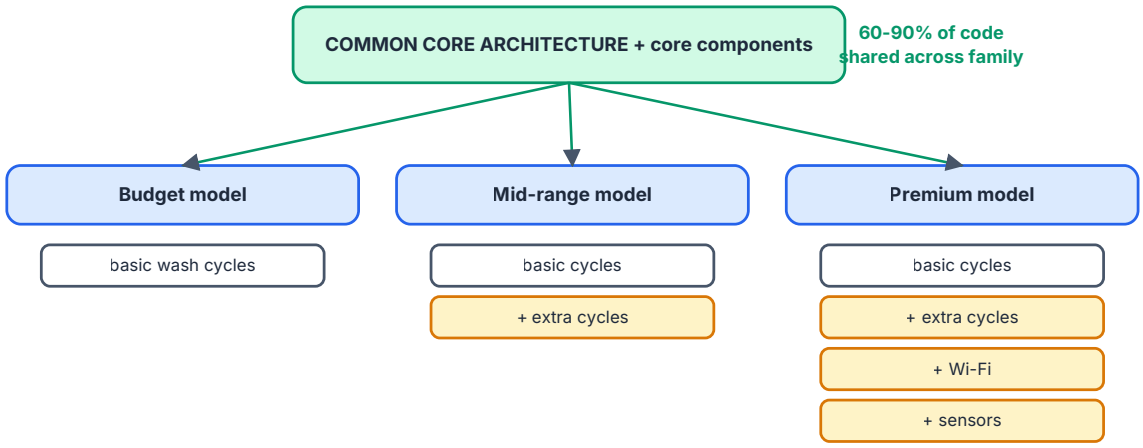
An online-banking app and a mobile-wallet app both reuse the same **Login** and **Payments** components; only the application-specific parts (statements, offers) differ.

📖 GFG: Component Based Software Engineering

2.2 Application Families Allow Significant Reuse

Definition

An **application family** (software product line) is a set of related systems sharing a **common core architecture and features** but differing in specific characteristics for different customers, markets or platforms.



Washing-machine family: products differ only at variation points (optional components)

Fig. 10 — One core architecture, three washing-machine models that differ only at variation points

Because members share a common architecture, common core components and differ only at well-defined **variation points**, reuse can reach **60–90 %** of the codebase — far higher than ad hoc component reuse.

Real-life use case — Example from the notes

Washing-machine embedded software: one core family; budget, mid-range and premium models differ in optional components (extra wash cycles, Wi-Fi, advanced sensors).

2.3 Developing Application Systems from Reusable Components

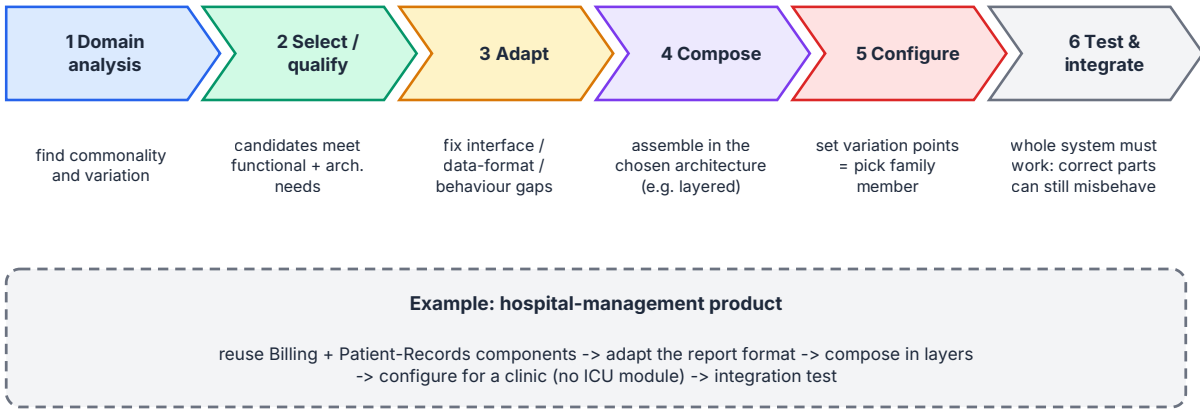


Fig. 11 — Six-step process from domain analysis to tested system

1. **Domain analysis** — commonalities and variations across the family.

2. **Component selection / qualification** — pick candidates meeting functional and architectural needs.
3. **Adaptation** — resolve interface, data-format or behaviour mismatches.
4. **Composition** — assemble per the chosen architecture (e.g. layered, §2.6).
5. **Configuration** — set variation points to get the desired family member.
6. **Testing & integration** — individually correct components can still misbehave together.

> ⭐ **Exam tip**
>
> Remember the order: **Analyse → Select → Adapt → Compose → Configure → Test**.

2.4 Reuse Variability

> 📘 **Definition**
>
> **Variability** is the ability of a reusable component or application family to be configured or adapted to different needs **without a full rewrite**.

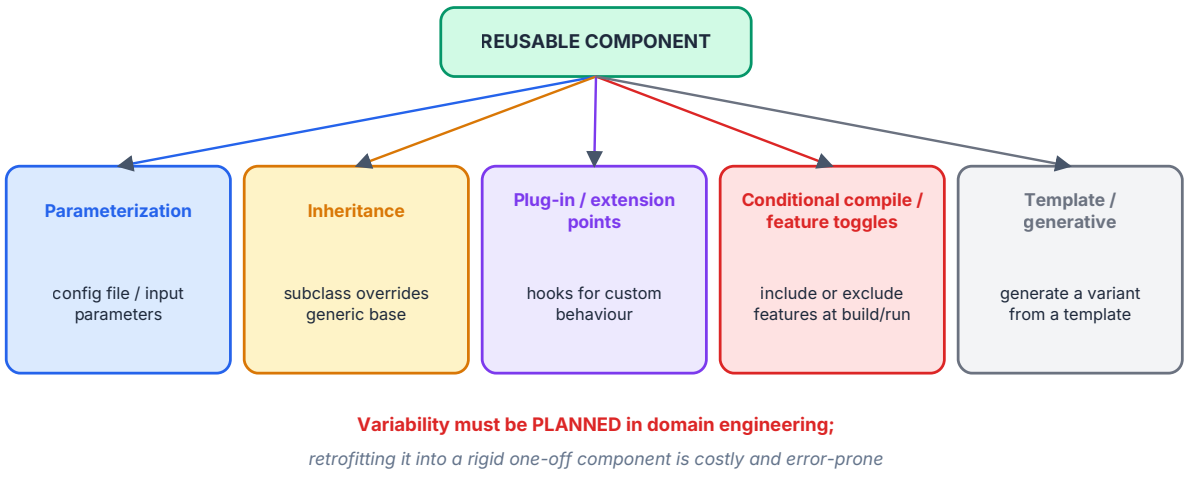


Fig. 12 — Five mechanisms to achieve variability

Mechanism	How
Parameterization	Behaviour set by input parameters or config files
Inheritance / specialization	Subclasses override or extend a generic base component
Plug-in / extension points	Explicit hooks where custom behaviour is injected without modifying the core
Conditional compilation / feature toggles	Include or exclude code paths at build time or run time
Template / generative techniques	Generate a tailored variant from a generic template

> ⚠️ **Careful**
>
> Variability must be **planned during domain engineering**. Retrofitting it into a rigid one-off component is far more expensive and error-prone.

🏭 Real-life use case — Real-life use

A CMS with a plugin system (WordPress): the core stays unchanged while each site adds plugins — **extension points** in action.

2.5 Facades for Component System Internals and Externals

📘 Definition

A **Facade** is a structural design pattern that gives a **simplified, unified interface** to a complex subsystem, hiding its internal complexity from clients.

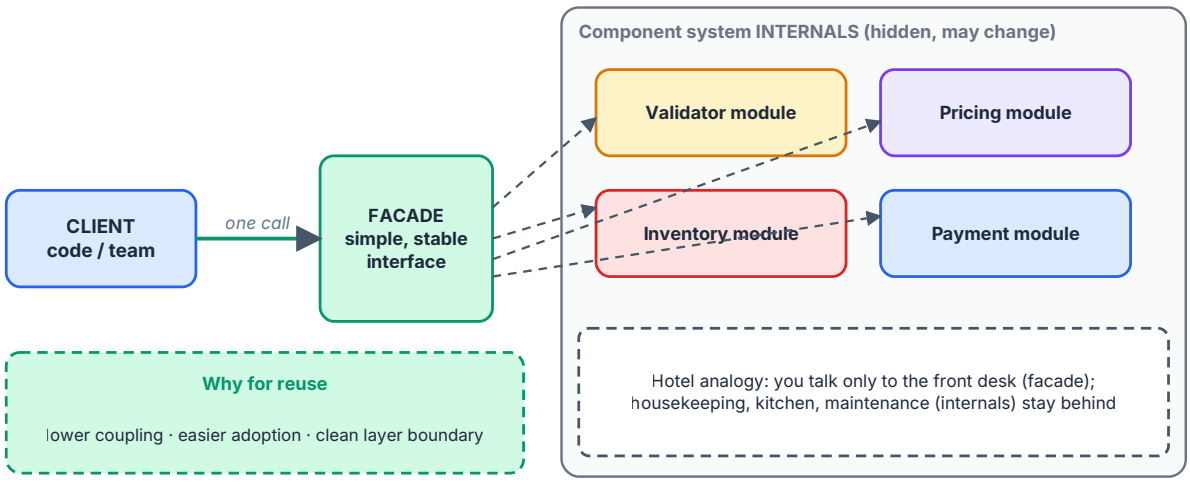


Fig. 13 — The client talks only to the facade; internals can change freely

- **Internals** — the real, often complex implementation (many classes / modules).
- **Externals (the facade)** — one clean, high-level interface exposed to consumers.

Why it matters for reuse	Explanation
Reduces coupling	Clients depend on the stable facade, so internals can change without breaking them
Simplifies usage	An approachable component is more likely to be adopted and reused
Enables layering	Facades sit at layer boundaries so each layer talks only to the facade below

🧠 Yaad rakho — hotel analogy

Guest sirf **front desk** (facade) se baat karta hai; housekeeping, kitchen aur maintenance (internals) se nahi.

📖 GFG: Facade Design Pattern

2.6 Organizing a System in Layered Architecture

📘 Definition

Layered (N-tier) architecture stacks a system into layers with distinct responsibilities; each layer communicates only with the layer directly above or below.

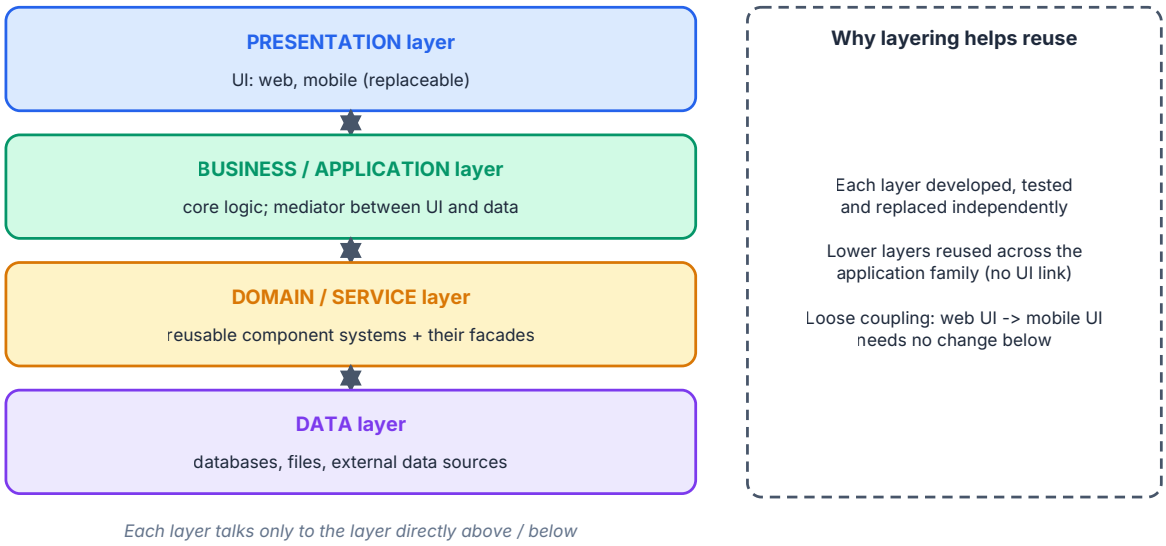


Fig. 14 — Classic 4-layer structure and why layering helps reuse

1. **Presentation layer** — UI, where users see and enter data.
2. **Business / application layer** — core business logic; mediates between presentation and data.
3. **Domain / service layer** — encapsulates reusable component systems and their facades (sometimes merged with business).
4. **Data layer** — persistence: databases, files, external data sources.

Why layering supports reuse	Explanation
Independent development	Each layer can be built, tested and replaced if its (facade) interface stays stable
Reusable lower layers	Data-access and shared business layers do not depend on any UI, so many applications in the family reuse them
Loose coupling	Switching web UI to mobile UI needs no change in business or data layers

🏭 Real-life use case — Real-life use

One backend (business + data layers) serves a React web app and a Flutter mobile app — only the presentation layer is written twice.

📖 GFG: Software Architectural Patterns · Layered Architecture

❓ Likely exam questions — Unit II

- Application system vs component system.
- What is an application family? Why does it allow 60–90 % reuse?
- Steps to develop an application from reusable components.
- What is variability? Explain its mechanisms.
- What is a facade? Why is it important for reuse? Draw it.
- Explain layered architecture and how it supports reuse.

UNIT III

Software Change, Evolution, Maintenance & Reengineering

Lehman's laws | Maintenance types, models, metrics | Reengineering process | Program comprehension | Reverse engineering | Restructuring | Forward engineering | Re-documentation | Software aging

3.1 Software Change & Software Evolution

Definition

Software change is any modification after initial deployment — fixing defects, adding features or adapting to new environments. **Software evolution** is the broader, long-term fact that systems must keep changing to remain useful.

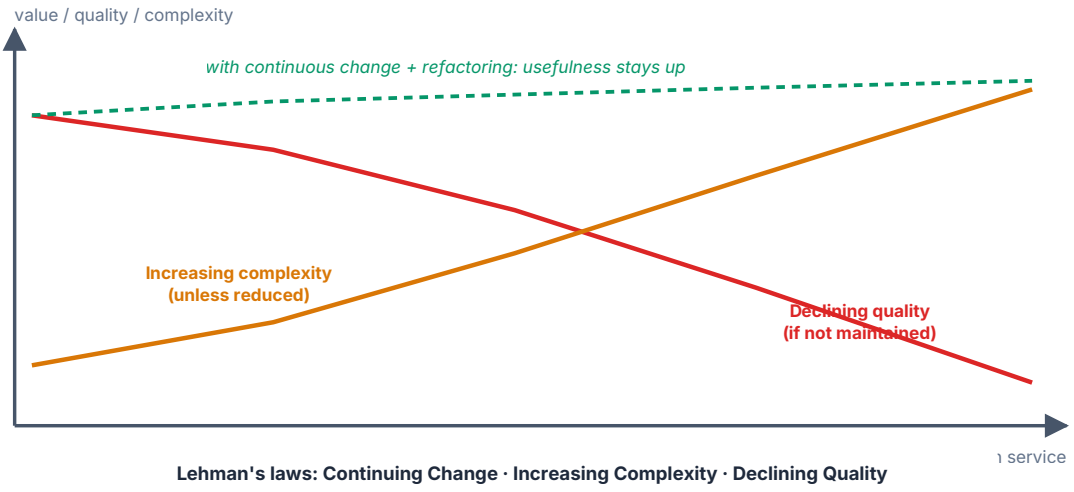


Fig. 15 — Lehman's laws: continuing change, rising complexity, falling quality

Lehman's law	Meaning
Continuing change	A system must be continually adapted or it becomes progressively less useful
Increasing complexity	Complexity grows as the system evolves unless work is done to reduce it
Declining quality	Perceived quality declines unless the system is rigorously maintained and adapted

The environment keeps changing (new regulations, hardware, user expectations), so evolution is an **inevitable, ongoing process** — not one-time bug fixing.

⭐ **Exam tip**

Lehman's laws are frequently tested — write the three above with one line each and draw the graph.

🧠 **Yaad rakho**

C-I-D: Change, Increasing complexity, Declining quality.

📖 GFG: Software Maintenance · Lehman's Laws

3.2 Software Maintenance: Types, Models and Metrics

📘 **Definition**

Software maintenance is modifying software after delivery to correct faults, improve performance or adapt it to a changed environment.

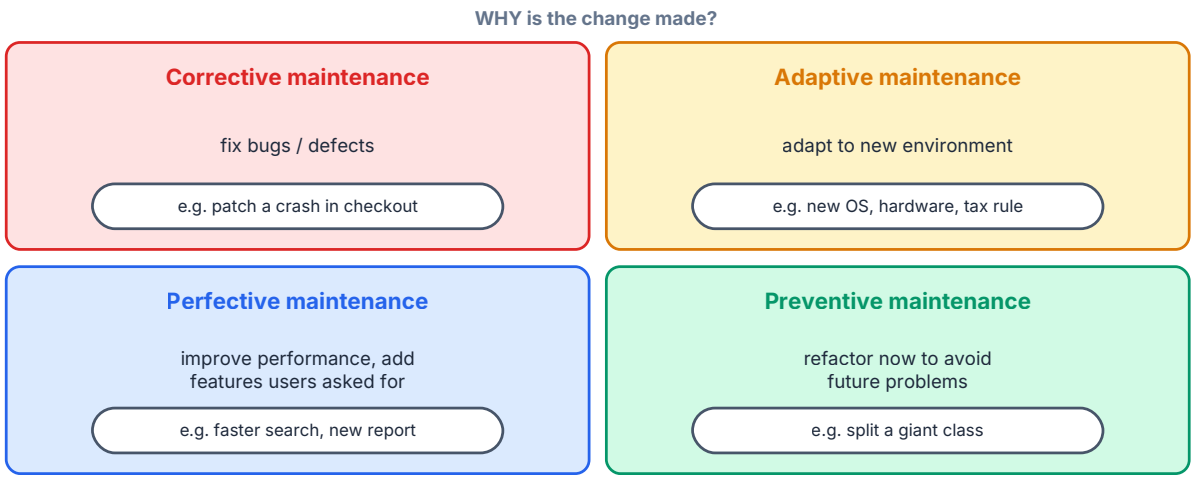


Fig. 16 — The four classic types of maintenance

Type	Purpose
Corrective	Fix discovered bugs / defects
Adaptive	Adapt to a changed environment — new OS, hardware, regulation
Perfective	Improve performance or maintainability, or add features users asked for
Preventive	Change now to prevent future problems (e.g. refactoring)

🧠 **Yaad rakho**

C-A-P-P — Corrective (bug), Adaptive (environment), Perfective (improve), Preventive (future).

Maintenance models

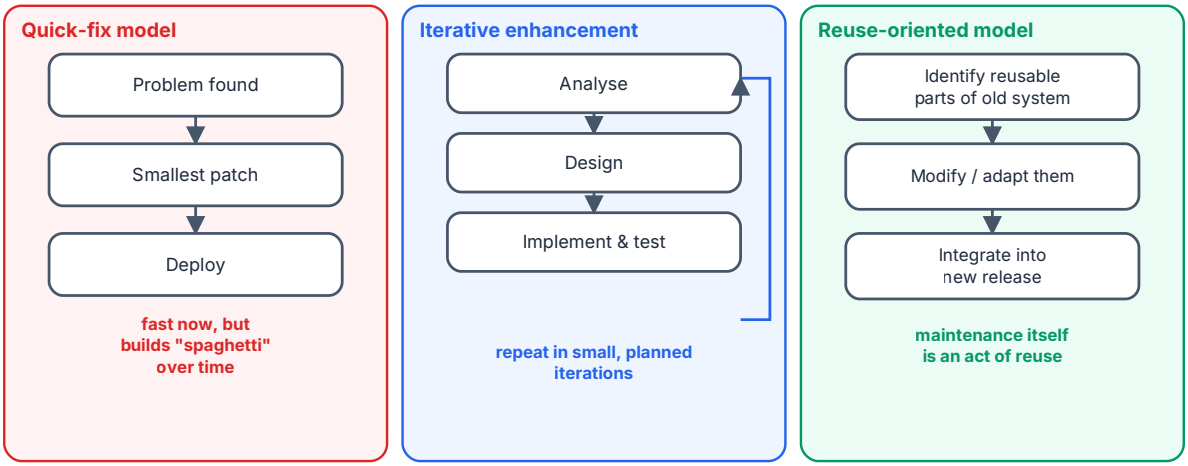


Fig. 17 — Quick-fix, iterative enhancement and reuse-oriented models

Model	Idea
Quick-fix	Smallest possible change for the immediate problem — fast, but risks "spaghetti code"
Iterative enhancement	Maintenance as small planned iterations, each with analysis, design and testing
Reuse-oriented	Maintenance as reuse of existing components with targeted modification (§1.1)

Maintenance metrics

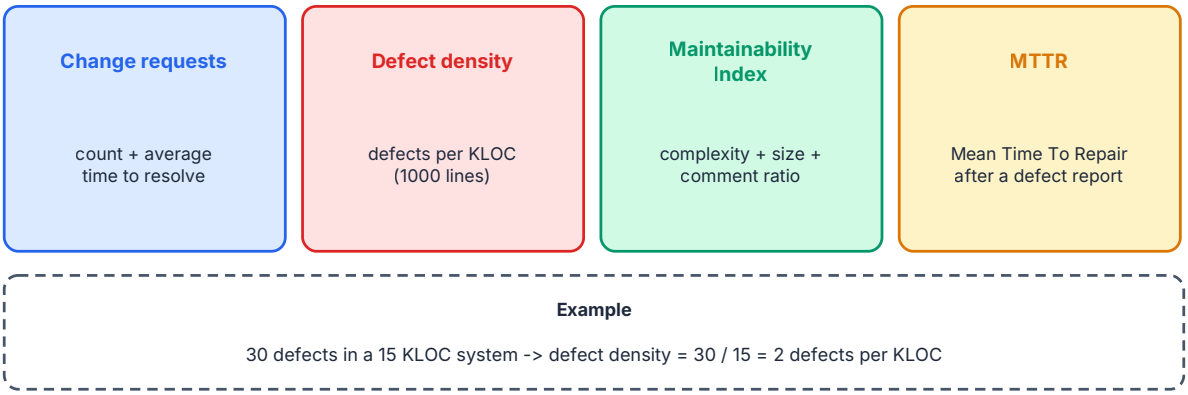


Fig. 18 — Four common maintenance metrics with a worked defect-density example

- **Change requests** — number received and average time to resolve.
- **Defect density** — defects per KLOC (thousand lines of code).
- **Maintainability Index** — composite of complexity, lines of code and comment ratio.
- **MTTR** — Mean Time To Repair: average time to fix a defect after it is reported.

🏭 Real-life use case — Real-life use

An app update that supports the new Android version is **adaptive**; fixing a crash is **corrective**; speeding up a slow screen is **perfective**; cleaning code before adding a big feature is **preventive**.

📖 GFG: Software Maintenance · Software Metrics

3.3 Reengineering: Process and Activities

📘 Definition

Software reengineering is the systematic examination and alteration of an existing system to reconstitute it in a new, improved form — without necessarily changing its overall functionality. Chosen over a rewrite when the system still has business value but is hard to maintain.

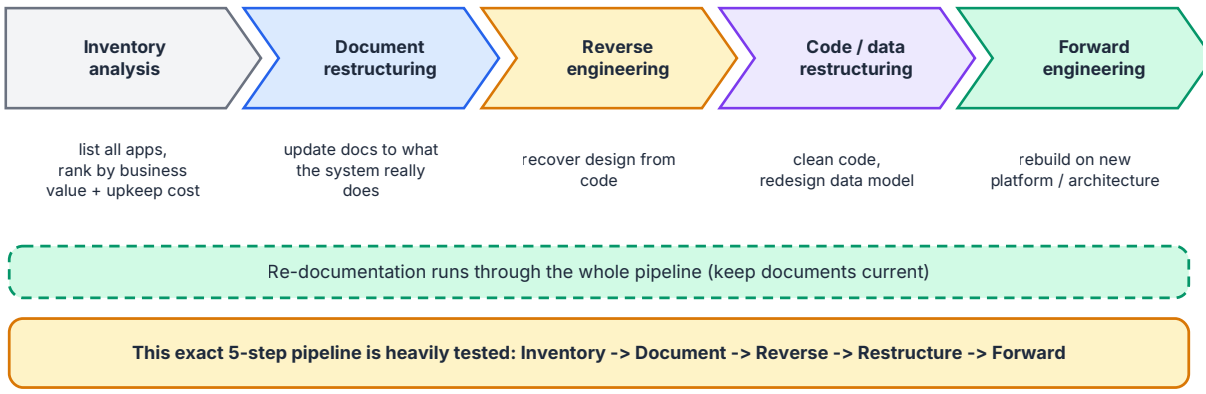


Fig. 19 — The five-step reengineering pipeline — heavily tested

⭐ Exam tip — Write this pipeline exactly

Inventory Analysis → Document Restructuring → Reverse Engineering → Code / Data Restructuring → Forward Engineering

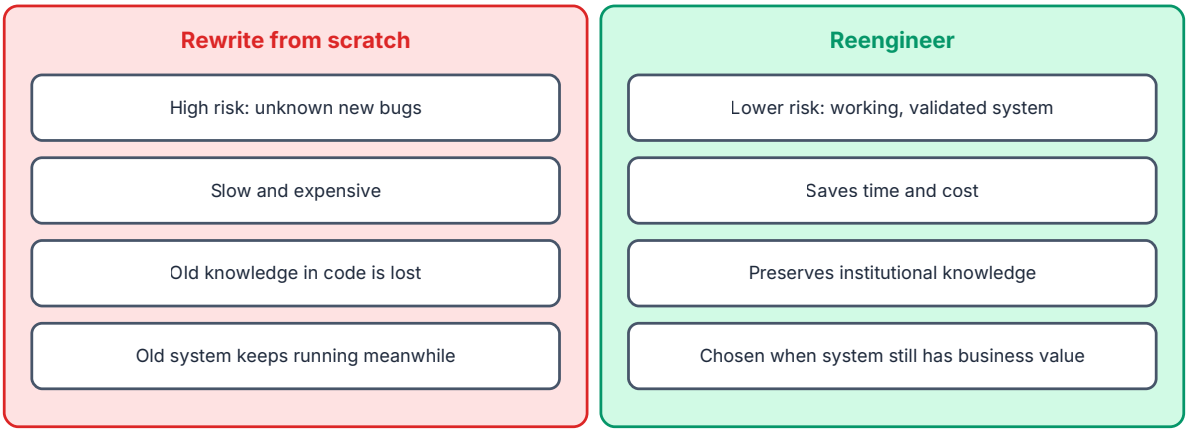


Fig. 20 — Why reengineer instead of rewriting from scratch

- **Lower risk** — a working system, validated by real use, already exists.
- **Saves time and cost** compared with ground-up redevelopment.
- **Preserves institutional knowledge** embedded in existing code and design.

🏭 Real-life use case — Real-life use

A bank moves a 25-year-old monolith to a layered / microservice design: it first recovers the design (reverse), cleans code and data (restructure), then rebuilds on the new stack (forward) — while the business keeps running.

📖 GFG: Software Re-Engineering

3.4 Program Comprehension

📘 Definition

Program comprehension is understanding an existing program's structure, behaviour and purpose — the prerequisite to any maintenance or reengineering, since you cannot safely change what you do not understand.

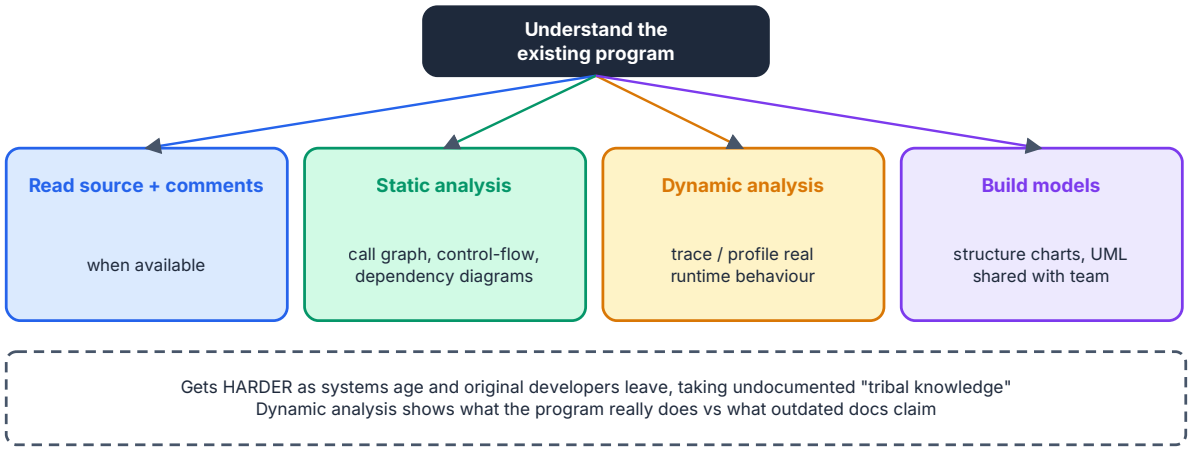


Fig. 21 — Four techniques to understand existing code

Technique	What it gives
Reading source & comments	First-hand view, when comments exist
Static analysis	Structure without running: call graphs, control-flow graphs, dependency diagrams
Dynamic analysis	Runtime behaviour via tracing / profiling — what it *actually* does vs outdated docs
Building models	Structure charts, UML to share understanding with the team

Comprehension gets much harder as systems age (§3.9) and original developers leave with undocumented "tribal knowledge".

3.5 Reverse Engineering

📘 Definition

Reverse engineering analyses an existing system to extract design and architecture information — working **backward from code** to recover higher-level abstractions that were lost or never documented.

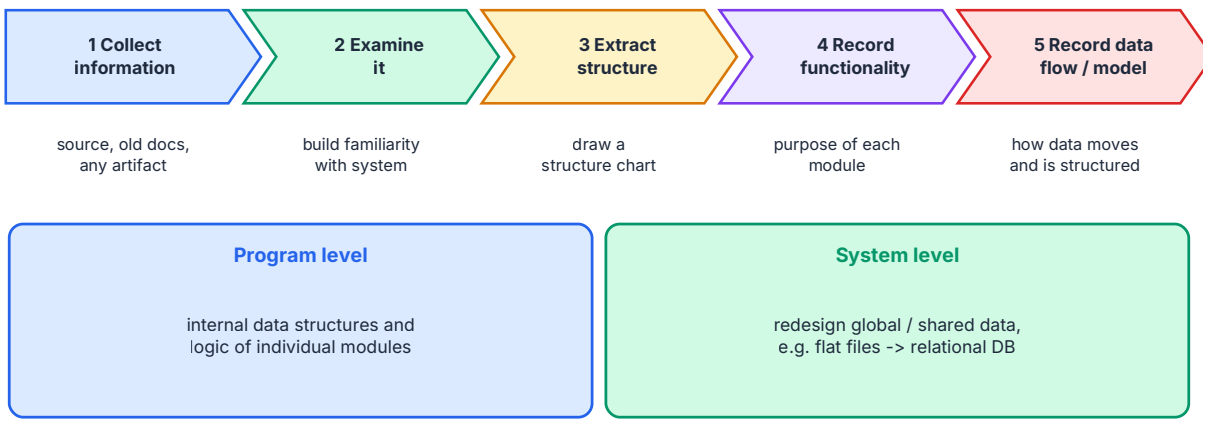


Fig. 22 — Five reverse-engineering steps and the two levels

Level	Meaning
Program level	Internal data structures and logic of individual modules
System level	Redesign global / shared data structures, e.g. flat files → relational or OO database

1. **Collect information** — source, old design docs, any documentation.
2. **Examine** the information to become familiar with the system.
3. **Extract structure** — usually drawn as a structure chart.
4. **Record functionality** of each module.
5. **Record data flow / data model** where applicable.

Usually the **first major activity** in reengineering — understand before restructuring or rebuilding.

📖 GFG: Reverse Engineering

3.6 Restructuring

📘 **Definition**

Restructuring transforms a system from one representation to another **at the same level of abstraction** while preserving its external functionality — it changes *how* it is organised, not *what* it does.

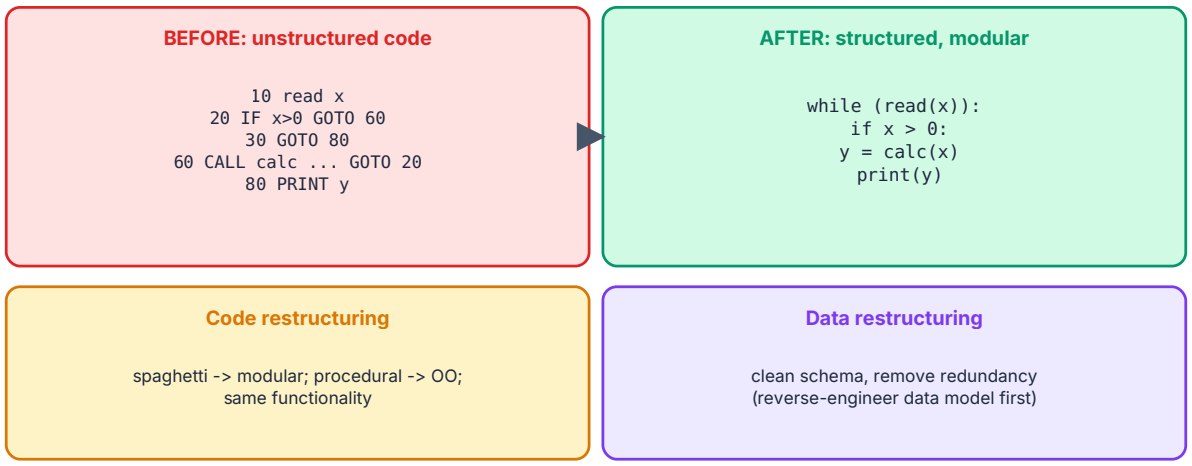


Fig. 23 — Code restructuring (spaghetti → structured) and data restructuring

Flavour	Meaning
Code restructuring	Improve quality — GOTO "spaghetti" → structured, modular code; procedural → object-oriented
Data restructuring	Reorganise data model / schema (often after reverse-engineering it) to remove redundancy and improve integrity

Lower risk than a rewrite because **observable behaviour stays constant**.

⚠️ **Careful**

Restructuring ≠ reverse engineering. Restructuring stays at the **same abstraction level**; reverse engineering moves **up** a level.

3.7 Forward Engineering

📘 **Definition**

Forward engineering is the traditional move from high-level, implementation-independent design to physical implementation — the regular direction: requirements → design → code.

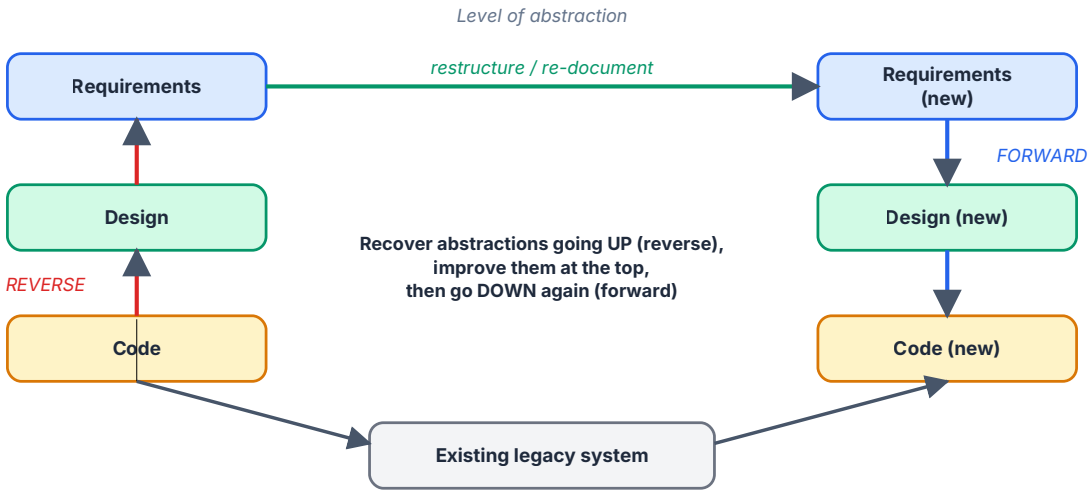


Fig. 24 — Horseshoe view: go UP with reverse engineering, DOWN with forward engineering

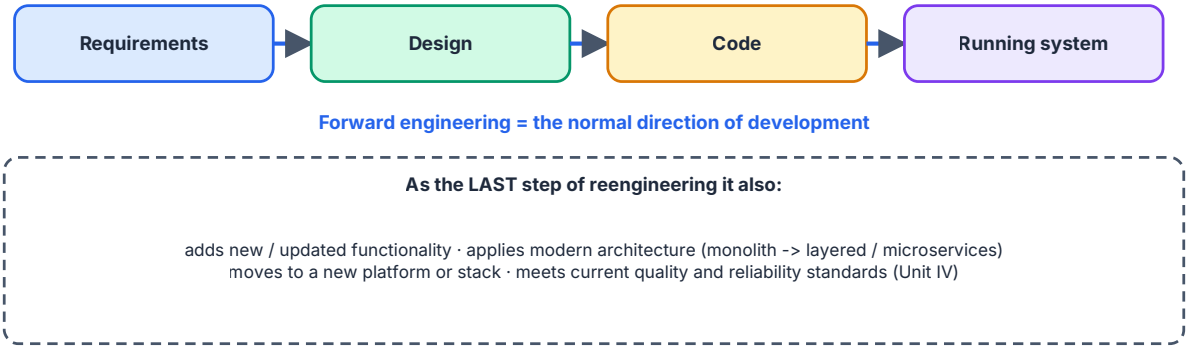


Fig. 25 — Forward engineering as the final step of reengineering

After reverse engineering has recovered the design and restructuring has cleaned code and data, forward engineering **rebuilds** a modernised system, often on a new platform or stack. It also:

- Adds new or updated functionality not in the original.
- Applies modern principles (monolith → layered / microservices).
- Aligns the system with current quality / reliability standards (Unit IV).

3.8 Re-documentation

📘 **Definition**

Re-documentation creates or revises documentation to reflect the system's **current state** — needed when docs are outdated, incomplete or never existed.

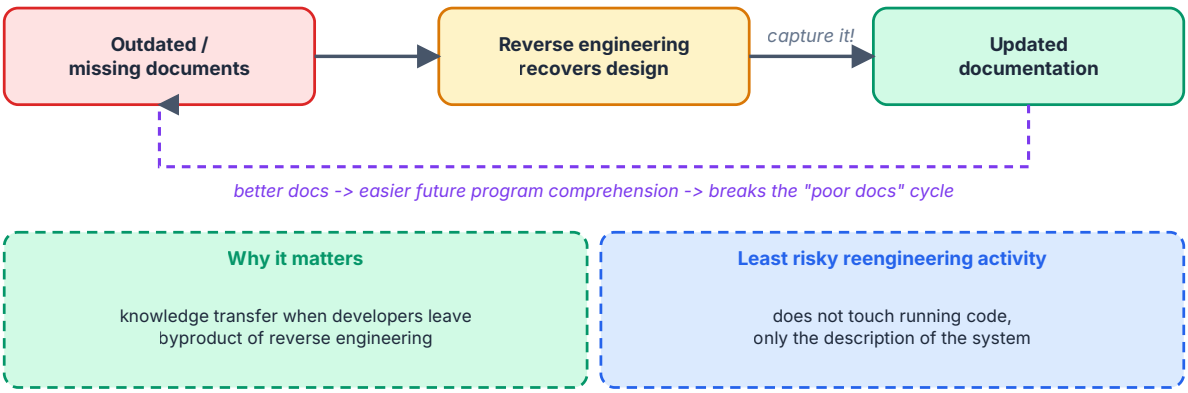


Fig. 26 — Breaking the vicious cycle of poor documentation

- Supports future comprehension — breaks the cycle "bad docs → hard maintenance → no time for docs".
- Captures the **by-product of reverse engineering** instead of losing it again.
- Improves knowledge transfer when original developers leave.

> ⭐ **Exam tip**
>
> Re-documentation is the **least risky** reengineering activity — it never touches running code.

3.9 Software Aging

> 📘 **Definition**
>
> **Software aging** is the degradation of a system's quality, performance and maintainability over its operational life — even without any new defect being introduced.

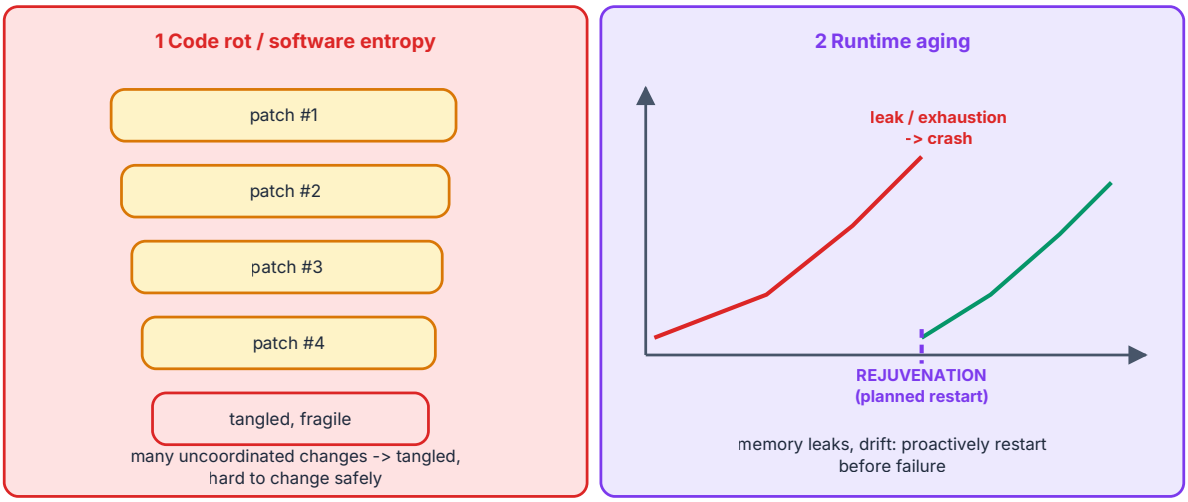


Fig. 27 — Two meanings of aging: code rot and runtime aging with rejuvenation

Meaning	Detail
1. Accumulated internal decay ("code rot", software entropy)	Many small uncoordinated changes over years tangle the structure and make it harder to change safely — links to Lehman's "increasing complexity"
2. Runtime aging (reliability engineering)	Long-running processes accumulate problems with no code change — memory leaks, resource exhaustion, numerical drift — causing slowdown or crashes

Mitigation for runtime aging: **software rejuvenation** — proactively and periodically restarting / resetting internal state **before** it fails.

🏭 **Real-life use case — Real-life use**

A server that must be restarted every few weeks because memory keeps growing — scheduled rolling restarts are **rejuvenation**.

🧠 **Yaad rakho**

Aging = **code sadta hai (rot)** aur **process thakta hai (leak)**. Dawa: reengineering aur rejuvenation.

❓ **Likely exam questions — Unit III**

- Explain Lehman's laws of software evolution.
- Types of maintenance with examples (C-A-P-P).
- Maintenance models and metrics (defect density, MTTR, MI).
- Explain the reengineering process with a diagram. Reengineering vs rewrite.
- Program comprehension — static vs dynamic analysis.
- Reverse engineering: levels and steps.
- Code vs data restructuring.
- Forward engineering; reverse vs forward engineering.
- Why is re-documentation needed?
- Software aging and rejuvenation.

UNIT IV

Dependability, Reliability, Quality & CASE

Dependability | Safety | Availability | Reliability & models | Quality factors | V&V | SQA | CASE scope & SDLC | 2nd-gen CASE | CASE architecture

4.1 Software Dependability

📘 **Definition**

Dependability is the umbrella property describing the degree of trust that can justifiably be placed in a system's service.

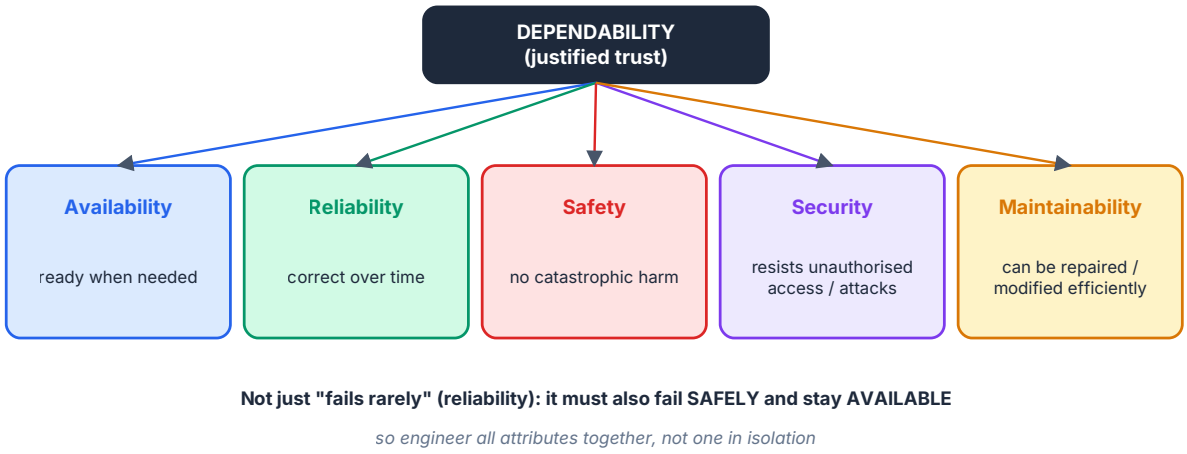


Fig. 28 — Dependability and its five attributes

Attribute	Meaning
Availability	System is ready for use when needed (§4.3)
Reliability	Performs correctly and continuously over time (§4.4)
Safety	No harm or catastrophe even when it fails (§4.2)
Security	Protects itself against unauthorised access / attacks
Maintainability	Can be efficiently modified / repaired

A dependable system must not only fail rarely (reliability) but also **fail safely** (safety) and stay up (availability) — so dependability needs a **holistic** approach, not optimising one attribute alone.

🧠 **Yaad rakho**

A-R-S-S-M: Availability, Reliability, Safety, Security, Maintainability.

📖 GFG: Reliability Attributes

4.2 Software Safety

📘 **Definition**

Software safety ensures that a system's failures do not lead to catastrophic outcomes — harm to people, the environment or major economic / property damage — even if the software is not 100 % reliable.

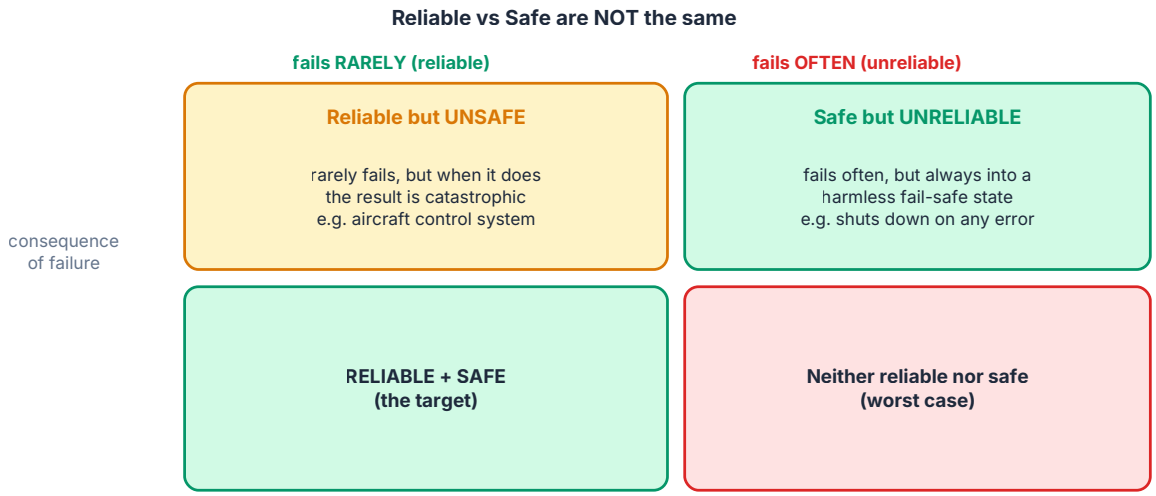


Fig. 29 — Reliable vs safe — the four combinations

Case	Example
Reliable but unsafe	Rarely fails, but failure is catastrophic — aircraft control system
Safe but unreliable	Fails often but always harmlessly (fail-safe) — shuts down safely on any error

⭐ **Exam tip — Frequently tested**

Reliability and safety are not the same thing. Always give both examples above.

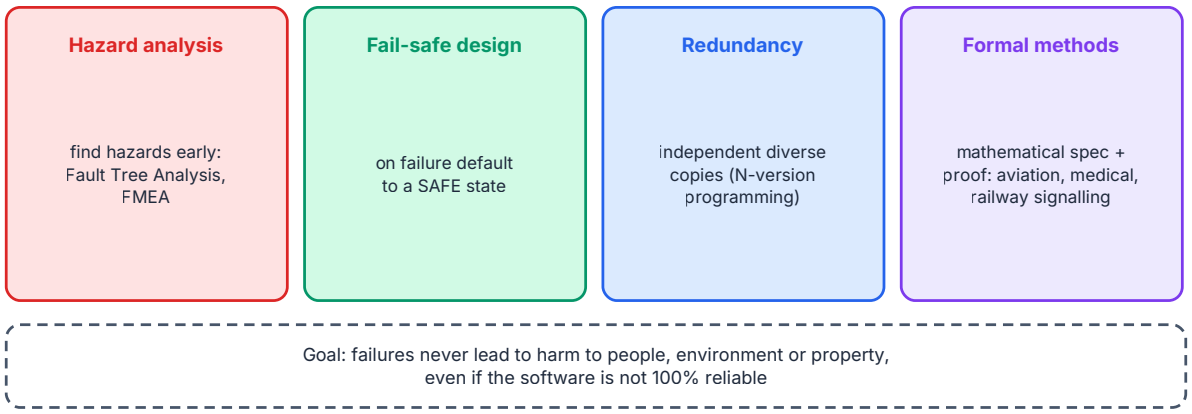


Fig. 30 — Four safety engineering techniques

1. **Hazard analysis** — identify hazards and causes early (Fault Tree Analysis, FMEA).
2. **Fail-safe design** — default to a safe state on failure.
3. **Redundancy** — multiple independent (often diverse, "N-version programming") components.
4. **Formal methods** — mathematical specification / verification for safety-critical parts (aviation, medical, railway signalling).

4.3 Software Availability

📘 **Definition**

Availability is the probability that a system is operational and able to deliver service at a given time, expressed as a percentage ("99.9 % — three nines").

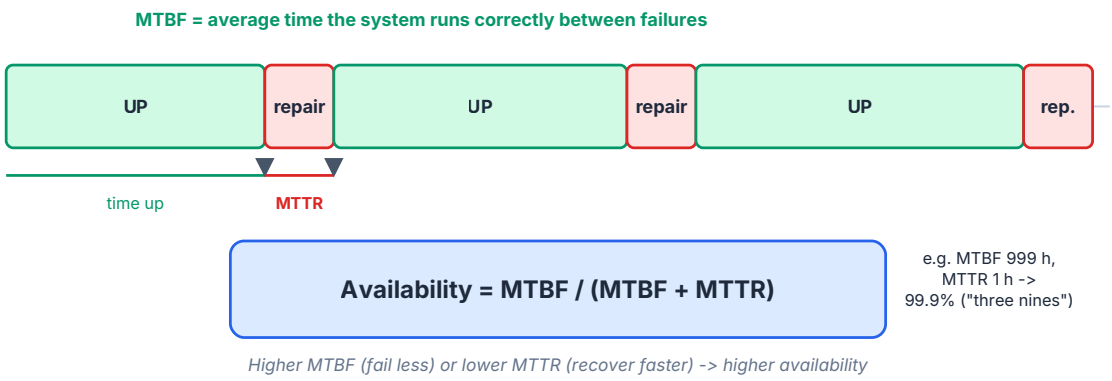


Fig. 31 — MTBF, MTTR and the availability formula with a worked example

⭐ **Exam tip — Most commonly tested formula**

Availability = MTBF / (MTBF + MTTR) — MTBF = average time running correctly between failures; MTTR = average time to detect and fix a failure.

Improve availability by **increasing MTBF** (fewer failures — reliability engineering) or **decreasing MTTR** (faster detection and recovery — monitoring, automated failover, redundancy).

🏭 Real-life use case — Real-life use

Cloud providers promise "99.99 %" uptime: they cut MTTR with health checks and automatic failover to a standby server.

📖 GFG: Reliability Testing — MTBF, MTTF, MTTR

4.4 Software Reliability: Metrics, Approaches and Models

📘 Definition

Software reliability is the probability that software performs its intended function without failure, for a specified period of time, under specified conditions.

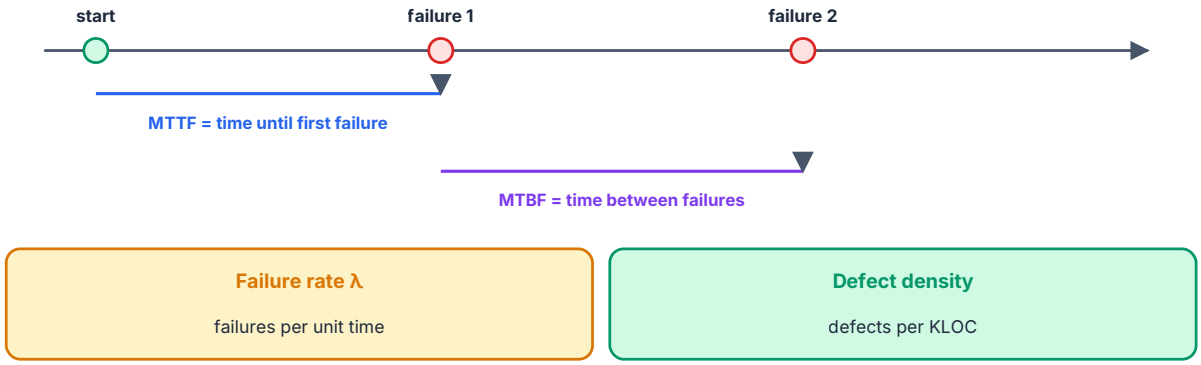


Fig. 32 — MTTF, MTBF and failure rate on a failure timeline; defect density

Metric	Meaning
MTTF	Mean Time To Failure — average time until the first failure
MTBF	Mean Time Between Failures — average gap between consecutive failures
Failure rate (λ)	Failures per unit time
Defect density	Defects per unit code size (per KLOC)

Approaches to achieve reliability

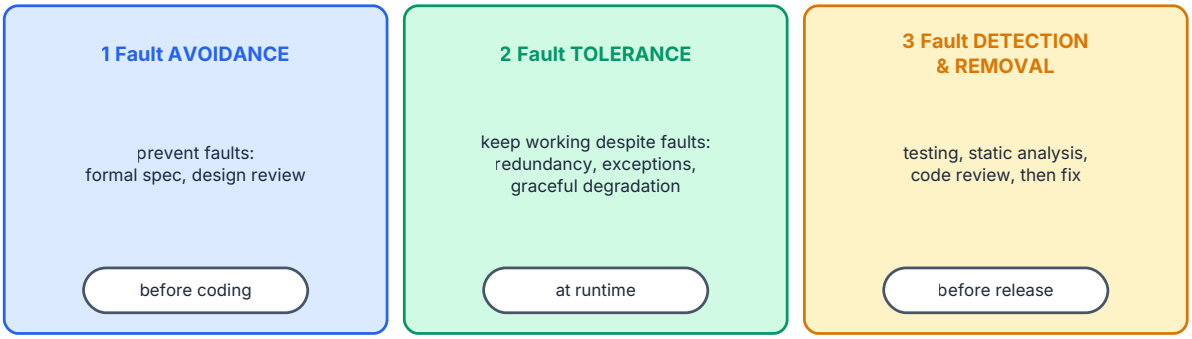


Fig. 33 — Fault avoidance, fault tolerance, fault detection & removal

1. **Fault avoidance** — rigorous practices (formal specification, design review) so faults are never introduced.
2. **Fault tolerance** — keep operating correctly despite faults (redundancy, exception handling, graceful degradation).
3. **Fault detection & removal** — testing, static analysis, code review before release.

Reliability models

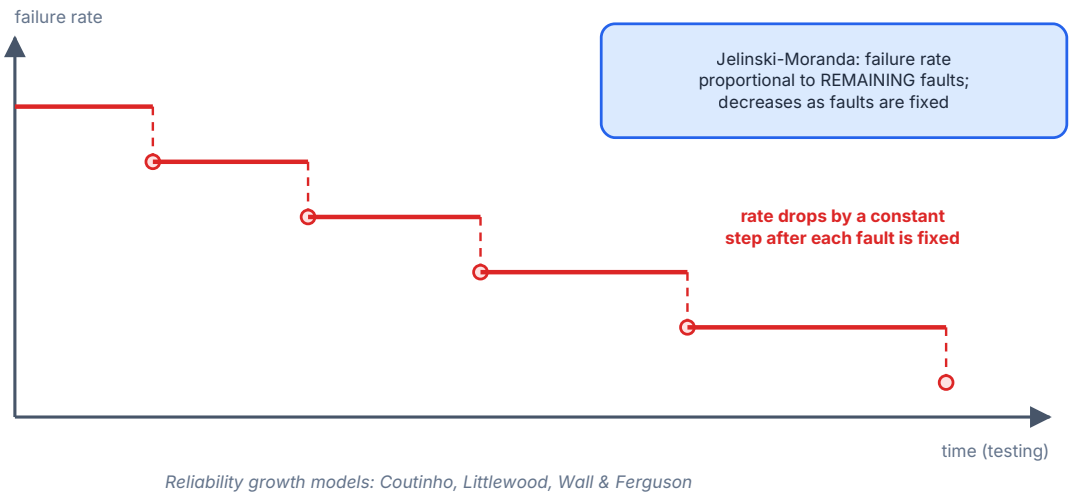


Fig. 34 — Jelinski-Moranda: failure rate drops by one step each time a fault is fixed

Model	Idea
Jelinski-Moranda	Early, well-known reliability **growth** model. Failure rate is **proportional to the number of remaining faults** and decreases as faults are found and fixed.
Reliability growth models (general)	Model how reliability improves during testing as defects are fixed — includes **Coutinho**, **Littlewood** and **Wall & Ferguson** models

⭐ **Exam tip — For numericals (standard JM form)**

With N initial faults and proportionality constant φ, the failure rate after i−1 faults are fixed is **$\lambda = \varphi \cdot (N - (i - 1))$**; mean time to the i-th failure = 1 / λ. Each fix lowers λ by the same step φ (see the figure).

📖 GFG: Jelinski-Moranda Model · Reliability Growth Models

4.5 Software Quality & Quality Factors

📘 Definition

Software quality is the degree to which a product meets specified requirements and satisfies user / customer needs.

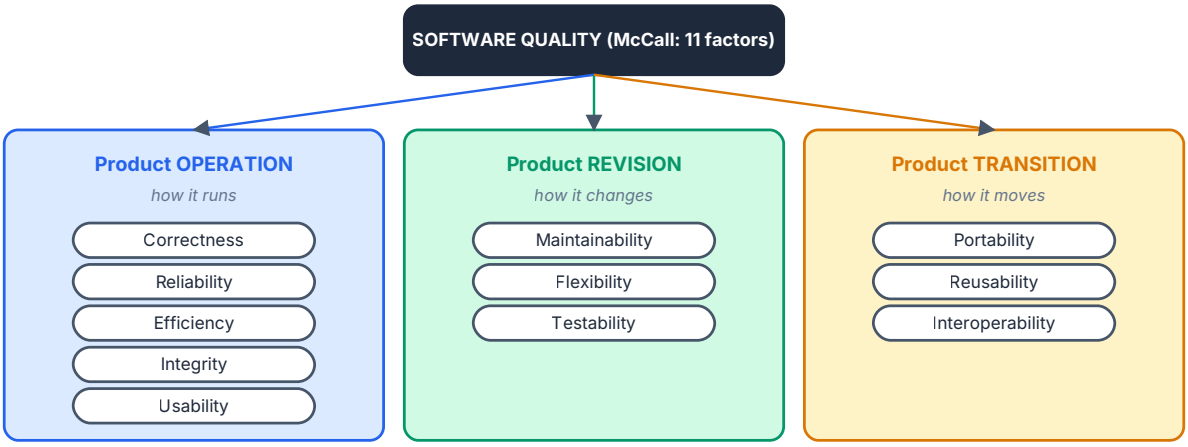


Fig. 35 — McCall's quality factors grouped into three categories

Category	Factors
Product operation	Correctness, Reliability, Efficiency, Integrity, Usability
Product revision	Maintainability, Flexibility, Testability
Product transition	Portability, Reusability, Interoperability

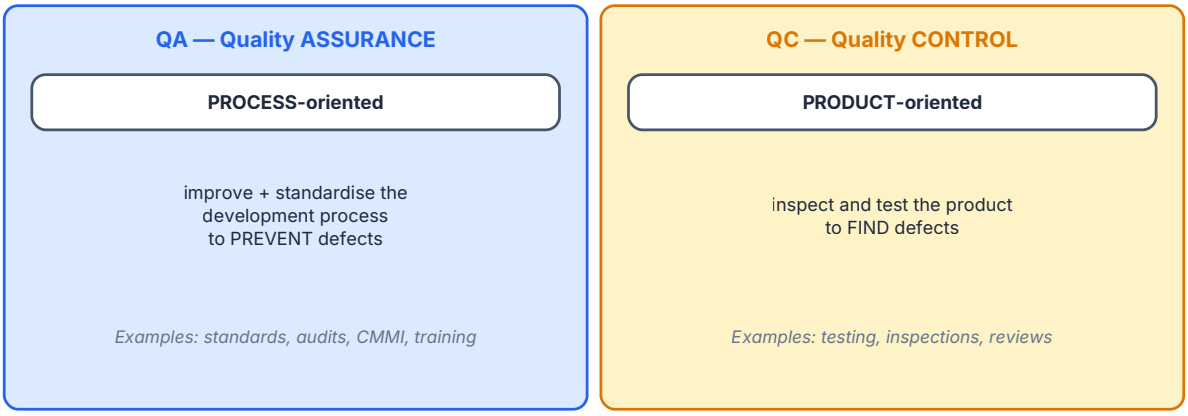


Fig. 36 — QA prevents defects in the process; QC finds defects in the product

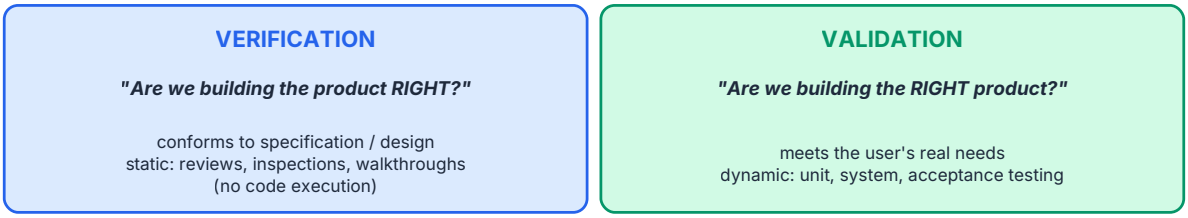
Quality Assurance (QA)	Quality Control (QC)
Process-oriented	**Product-oriented**
Improve and standardise the process to **prevent** defects	Find defects in the finished / in-progress product by inspection and testing

🧠 **Yaad rakho**

Operation 5, Revision 3, Transition 3 = 11 factors. QA = **process**, QC = **product**.

📖 GFG: Software Quality · Quality Metrics

4.6 Verification & Validation (V&V)



SDLC phase	Verification	Validation
Requirements	review for clarity	reflect real user needs
Design	design reviews	suits real world
Implementation	code reviews	unit testing
Testing	proper test cases exist	system + acceptance

Fig. 37 — "Building the product right" vs "building the right product"

Verification	Validation
"Are we building the product right?" Conforms to specification / design — static reviews, inspections, walkthroughs, no execution	**"Are we building the right product?"** Satisfies the user's real needs — dynamic testing (unit, system, acceptance)

V&V runs through the **whole SDLC**, not just at the end.

Phase	Verification	Validation
Requirements	Review documents for clarity / completeness	Check they reflect real user needs
Design	Design reviews / inspections	Check design suits real-world implementation
Implementation	Code reviews, walkthroughs	Unit testing
Testing	Ensure proper test cases exist	System and acceptance testing

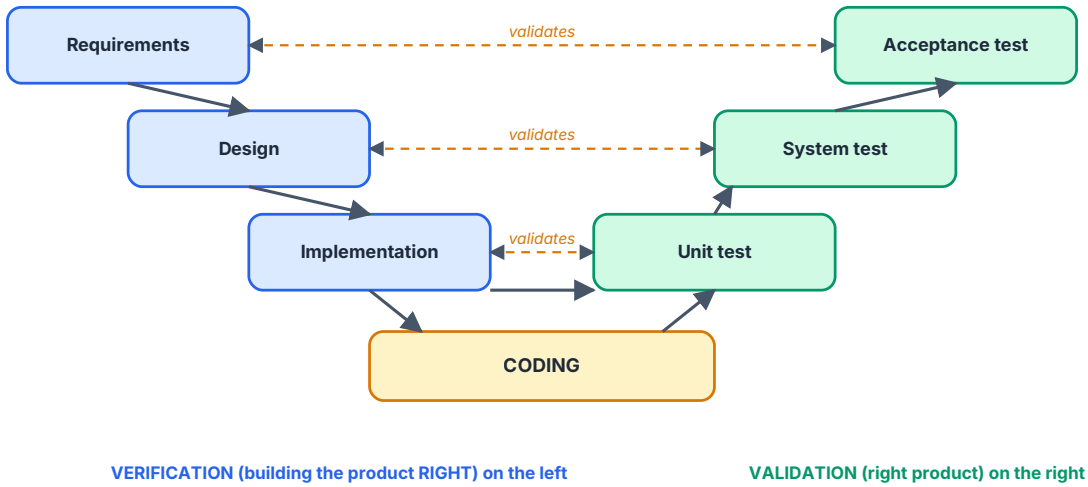


Fig. 38 — V-model: each verification phase on the left maps to a validation phase on the right

🧠 Yaad rakho

Verification = **"sahi bana rahe hain?"** (spec se match). Validation = **"sahi cheez bana rahe hain?"** (user ki zaroorat).

📖 GFG: Verification vs Validation · V-Model

4.7 Software Quality Assurance (SQA)

📘 Definition

SQA is a planned, systematic set of activities ensuring that software processes and products conform to established standards and procedures — the umbrella within which V&V, testing, reviews and metrics operate.

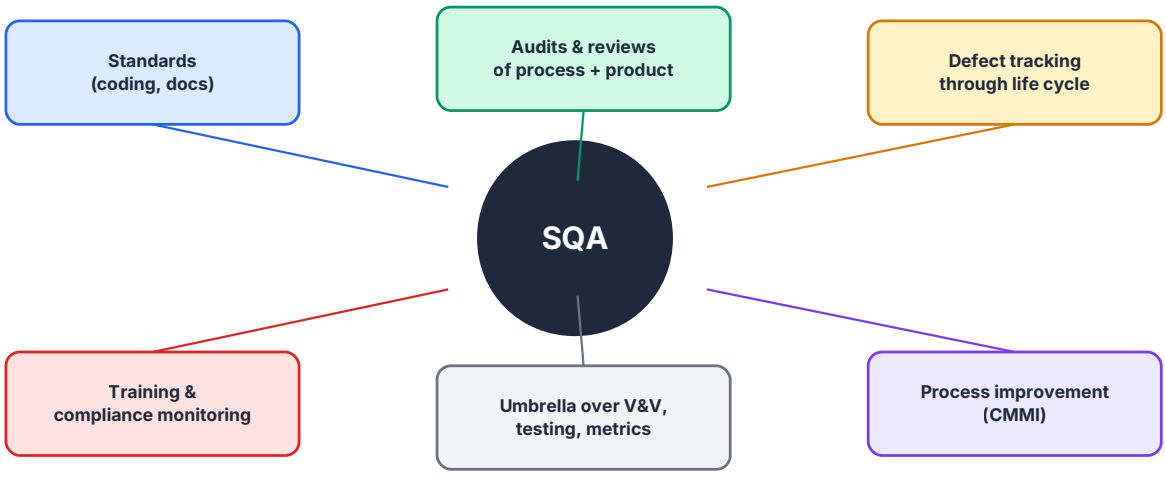


Fig. 39 — SQA as an umbrella over standards, audits, defect tracking, process improvement and training

- **Standards** — establish and enforce coding and documentation standards.
- **Audits & reviews** of process and product at defined milestones.

- **Defect tracking** through its life cycle.
- **Process improvement** with models like **CMMI**.
- **Training & compliance monitoring.**

SQA is fundamentally **preventive** (build quality in) rather than purely detective — broader than testing alone.

4.8 CASE: Scope and Technology

Definition

CASE (Computer-Aided Software Engineering) is the use of automated tools to support software development across the whole life cycle — requirements, design, coding, testing and maintenance.

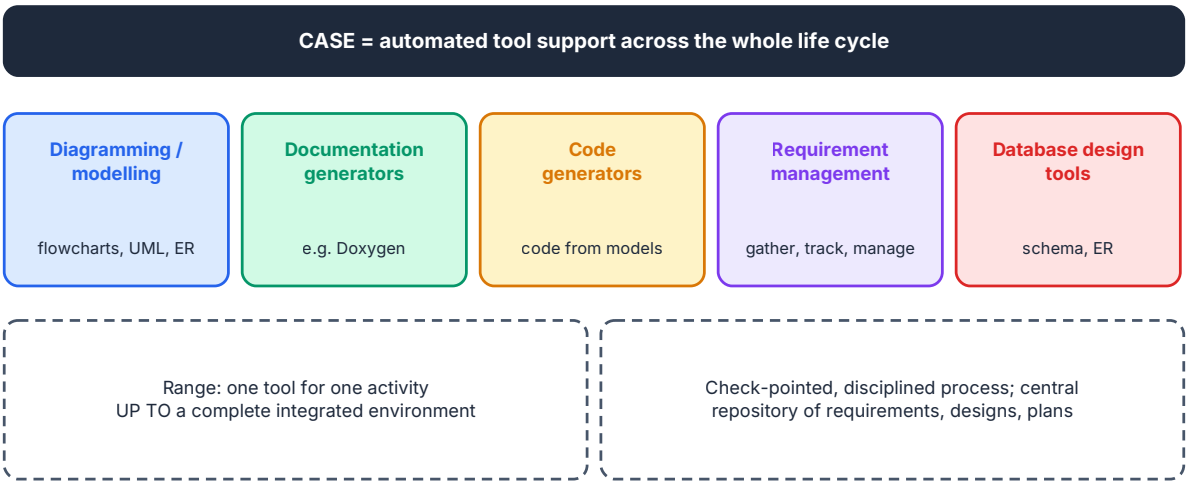


Fig. 40 — Scope of CASE and its technology categories

- Broad term: from a **single tool** (a diagramming tool) up to a **complete integrated environment**.
- Gives a **check-pointed, disciplined** approach so stakeholders can track milestones.
- Acts as a **central repository** for requirements, specifications, business plans, etc.

CASE technology	Use
Diagramming / modelling tools	Flowcharts, UML, ER diagrams
Documentation generators	Technical / user docs (e.g. Doxygen)
Code generators	Generate code from models
Requirement management tools	Gather, track and manage requirements
Database design / management tools	Schema design

📖 GFG: CASE · CASE Tool Scope

4.9 CASE Support in SDLC

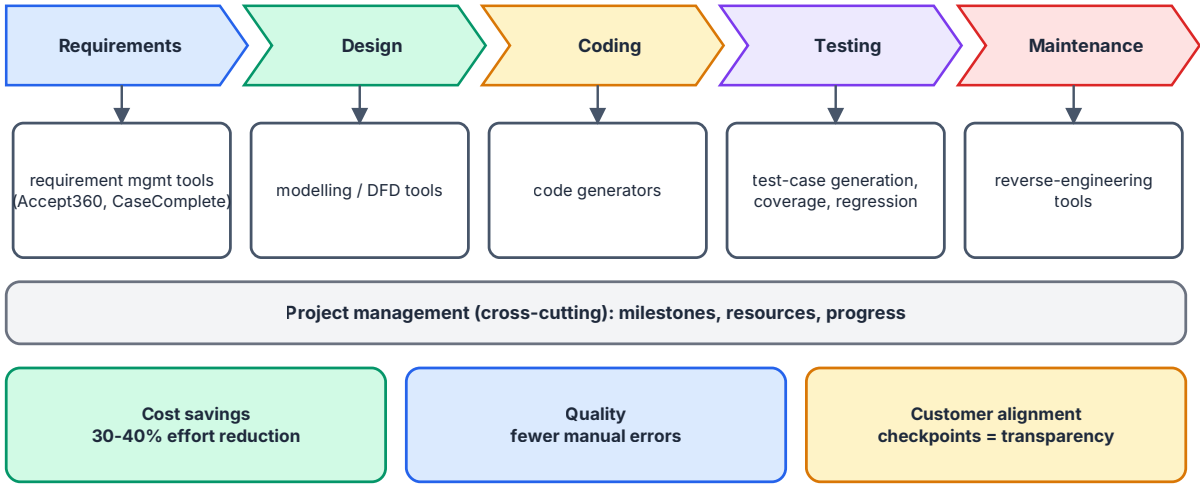


Fig. 41 — CASE tools supporting every SDLC phase, with project management across all

SDLC phase	CASE support
Requirements analysis	Requirement-management tools (Accept360, CaseComplete) to gather, track, validate
Design	Modelling / analysis tools for architecture, data-flow diagrams, behaviour
Coding	Code generators from designs — fewer manual errors
Testing	Automated test-case generation, code-coverage, regression testing
Maintenance	Reverse-engineering tools (Unit III) to understand and safely modify code
Project management *(cross-cutting)*	Track milestones, resources, progress

- **Cost savings** — studies estimate **30–40 %** effort reduction.
- **Quality enhancement** — automation reduces human error.
- **Customer alignment** — checkpoints keep the process transparent.

4.10 Second Generation CASE Tools

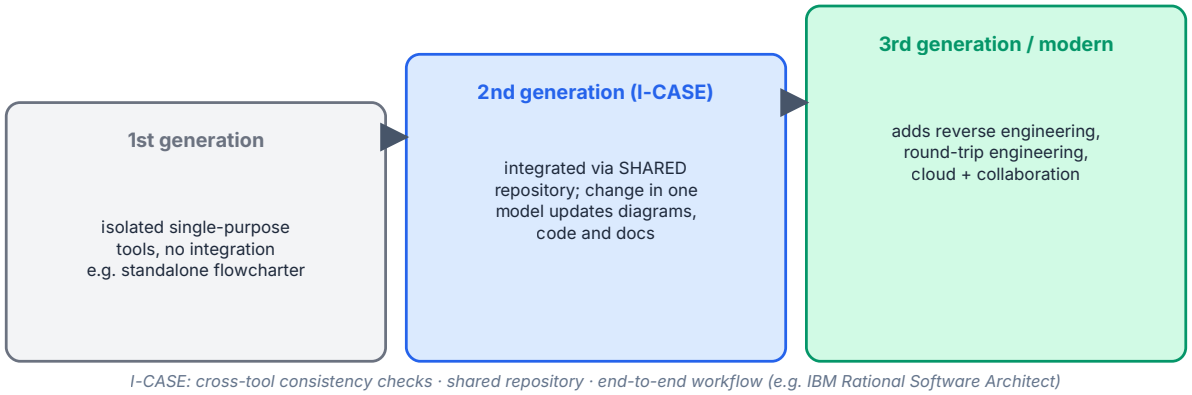


Fig. 42 — From isolated tools to integrated I-CASE to modern round-trip environments

Generation	Description
First	Isolated, single-purpose tools (standalone flowchart tool, code editor); no integration
Second — I-CASE	Integrated across SDLC phases via a **shared central repository**: a change to a data model propagates to diagrams, generated code and docs. Adds **cross-tool consistency checking**, **shared repository**, **end-to-end workflow** support (e.g. IBM Rational Software Architect)
Third / modern	Adds reverse-engineering, **round-trip engineering** (code ↔ models in sync both ways) and cloud / collaborative features

4.11 Architecture of a CASE Environment

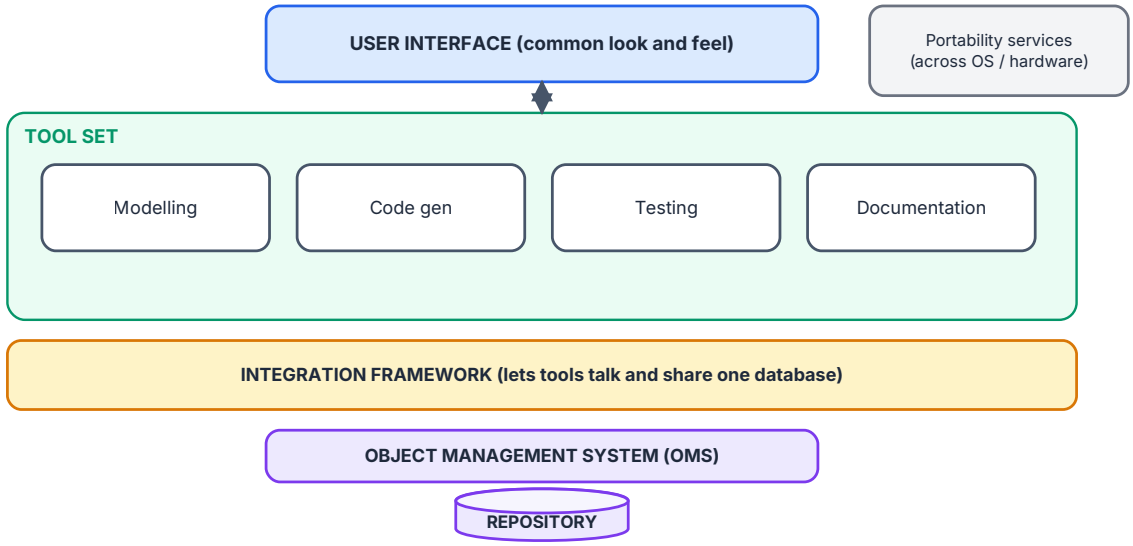


Fig. 43 — Four core elements plus integration framework and portability services

1. **User interface** — common front-end with a consistent look and feel.
2. **Tool set** — the individual tools (modelling, code generation, testing, documentation).

3. **Object Management System (OMS)** — controls access to the repository: storage, retrieval, versioning.
4. **Repository** — central store of all project artifacts for consistency and traceability.

- **Portability services** — let tools and the integration framework run across hardware / operating systems.
- **Integration framework** — specialised programs letting tools communicate and share the same project database.

🧠 **Yaad rakho**

U-T-O-R: User interface, Tool set, OMS, Repository.

📖 GFG: CASE Environment Architecture · Benefits of CASE

❓ **Likely exam questions — Unit IV**

- What is dependability? Explain its attributes.
- Reliability vs safety (with examples); safety engineering techniques.
- Availability formula; numerical on MTBF / MTTR.
- Reliability metrics; fault avoidance, tolerance, detection.
- Jelinski-Moranda model; reliability growth models.
- McCall quality factors; QA vs QC.
- Verification vs validation; V-model.
- Explain SQA and its activities.
- Scope and technology of CASE; CASE support in SDLC.
- First vs second generation CASE (I-CASE).
- Architecture of a CASE environment.

UNIT V

Usability Engineering & Advanced Paradigms

HCI & usability | Types of UI | Component-based GUI | Usability process & methods | AOSE | Cleanroom | Crowdsourcing | AI/ML in SDLC

5.1 Usability Engineering & HCI

Definition

Human-Computer Interaction (HCI) is the study of the design, evaluation and implementation of interactive computing systems for human use — how humans and computers communicate through an interface.

Definition — Usability (ISO 9241)

"The **effectiveness, efficiency and satisfaction** with which specified users achieve specified goals in a particular environment."

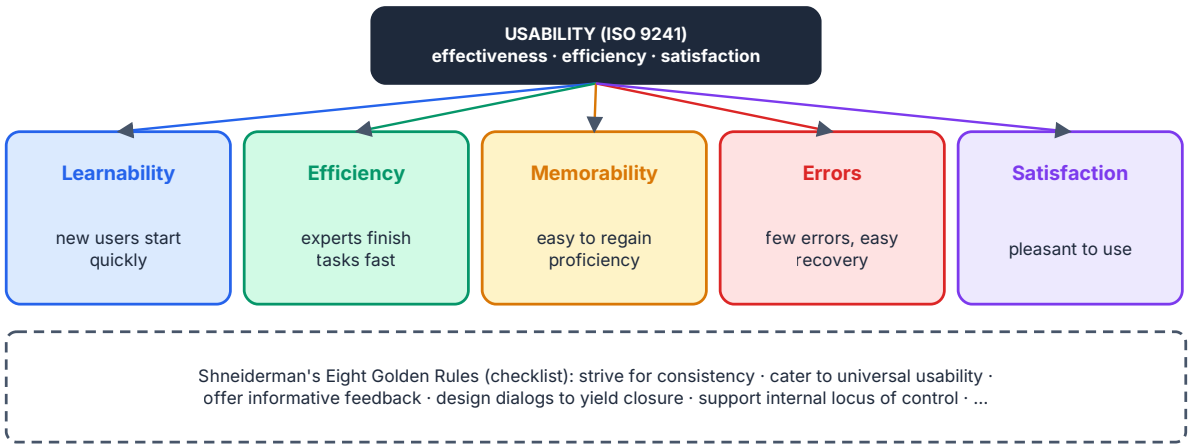


Fig. 44 — The five usability attributes

Attribute	Meaning
Learnability	How easily new users start using the system effectively
Efficiency	How quickly experienced users accomplish tasks
Memorability	How easily users regain proficiency after a break
Errors	How often users err and how easily they recover
Satisfaction	How pleasant the system is to use

Shneiderman's Eight Golden Rules (commonly tested checklist) include: strive for **consistency**, cater to **universal usability**, offer **informative feedback**, design dialogs to yield **closure**, support **internal locus of control**, among others.

> 🧠 **Yaad rakho**
>
> **L-E-M-E-S**: Learnability, Efficiency, Memorability, Errors, Satisfaction.

📖 GFG: HCI · HCI Guidelines · Usability Principles

5.2 Types of UI

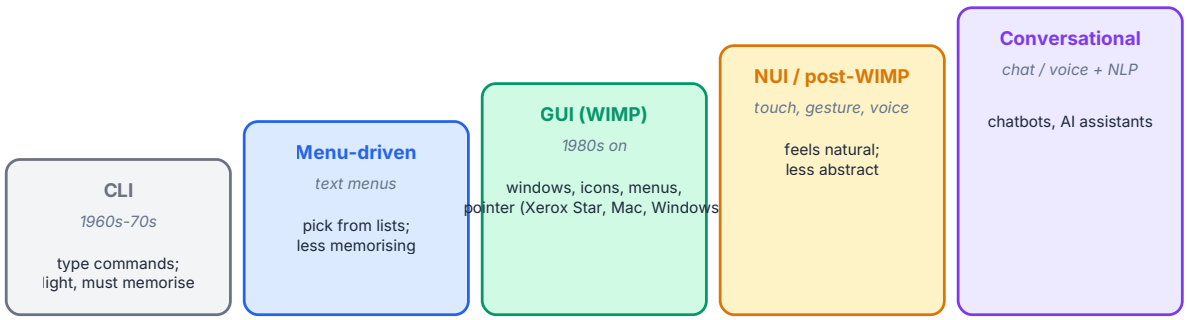


Fig. 45 — Evolution of user interfaces: CLI → menu → GUI → NUI → conversational

#	Interface	Detail
1	**CLI**	Type text commands; lightweight, low memory, but commands must be memorised (1960s–70s)
2	**Text-based / menu-driven**	Lists of textual options; less memorisation than pure CLI
3	**GUI**	Windows, Icons, Menus, Pointers (**WIMP**); dominant from the 1980s (Xerox Star, Apple, Microsoft)
4	**NUI / post-WIMP**	Touch, gesture, voice — feel more "natural" (smartphone touchscreens, voice assistants)
5	**Conversational / chat**	Text or voice with natural-language processing (chatbots, AI assistants)

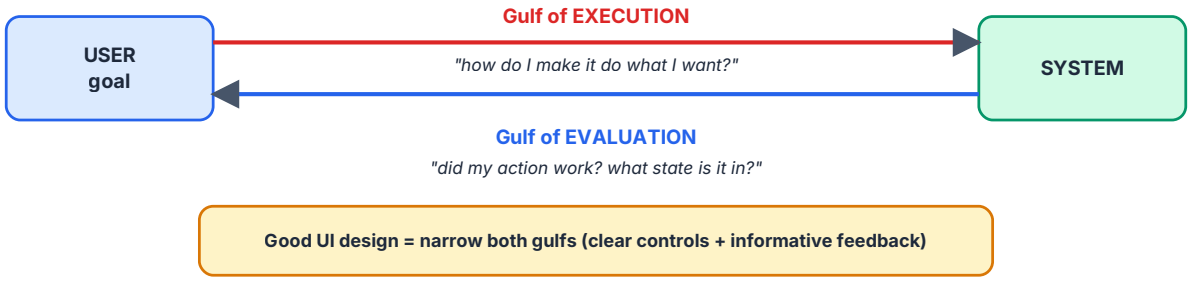


Fig. 46 — Gulf of execution and gulf of evaluation — each UI generation narrows them

- **Gulf of execution** — gap between what the user wants to do and how they must act.
- **Gulf of evaluation** — how easily the user can tell whether the action succeeded.

5.3 Component-Based GUI Development

Definition

Applying **CBSE** (§1.7) to GUIs: assemble interfaces from a library of **pre-built reusable widgets** — buttons, text fields, dropdowns, data grids, modals — instead of coding every element from scratch.

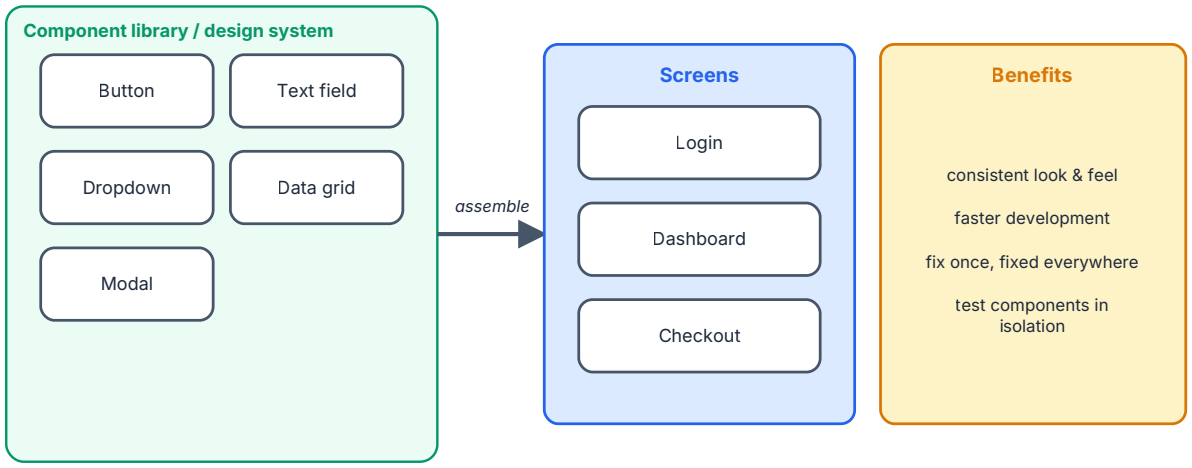


Fig. 47 — A component library assembled into screens

Benefit	Explanation
Consistency	Same components give uniform look and feel — the usability "consistency" principle
Faster development	Assembling screens from a design system beats custom-coding each one
Easier maintenance	Fix or restyle a shared component once; it propagates everywhere
Testability	Unit-test each component in isolation before assembling screens

🏭 **Real-life use case — Real-life use**

Material UI, Bootstrap or a company's internal design system — one shared Button used on every page of every product.

5.4 Usability Engineering Process and Methods

📘 **Definition**

Usability engineering is a structured, engineering-style approach that treats usability as a **measurable, testable requirement**, not a vague aesthetic goal.

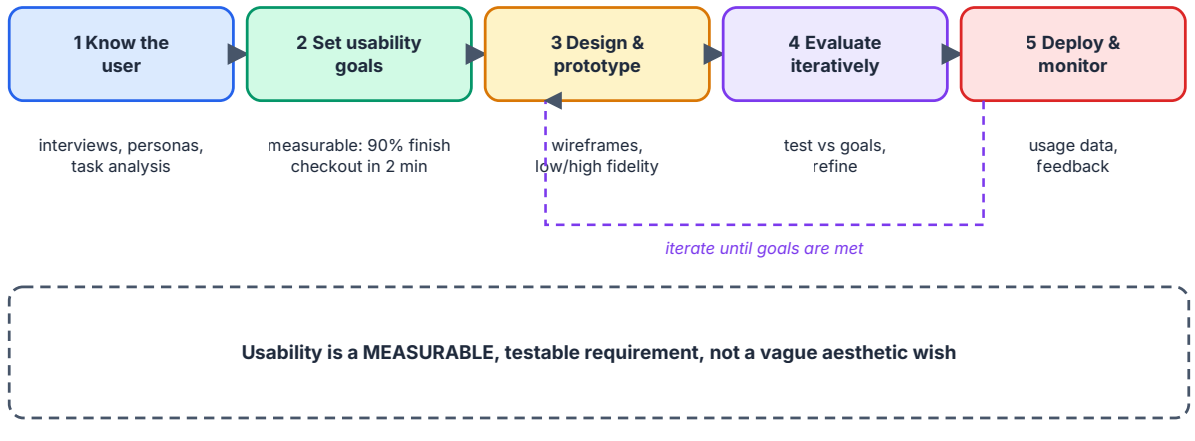


Fig. 48 — Five-step usability engineering process

1. **Know the user** — interviews, surveys, personas, task analysis.
2. **Set usability goals** — measurable, e.g. "90 % of new users complete checkout in 2 min without help".
3. **Design and prototype** — low- and high-fidelity prototypes / wireframes.
4. **Evaluate iteratively** — test against the goals and refine.
5. **Deploy and monitor** — keep collecting usage data and feedback after release.

Evaluation methods

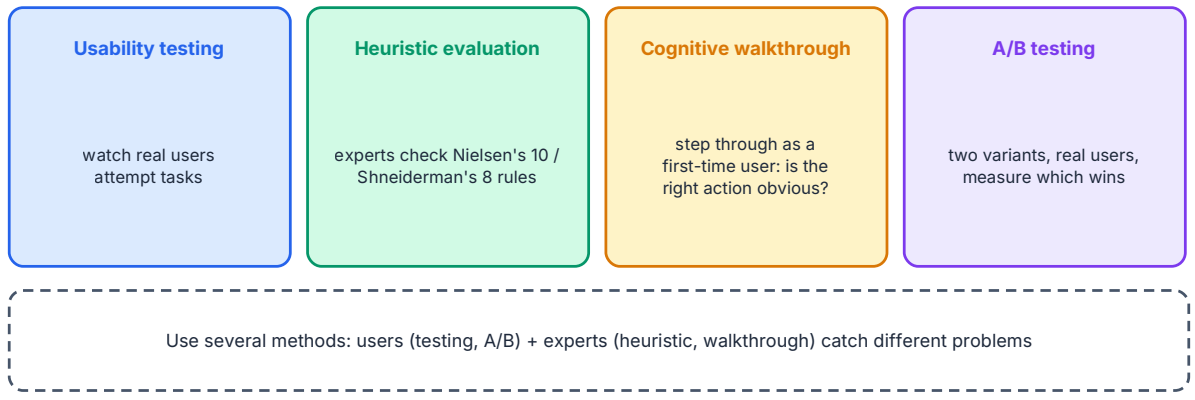


Fig. 49 — Four key evaluation methods

Method	Idea
Usability testing	Observe real users attempting tasks
Heuristic evaluation	Experts check the UI against heuristics (Nielsen's 10, Shneiderman's 8 rules)
Cognitive walkthrough	Step through tasks as a first-time user; is the right action obvious at each step?
A/B testing	Compare two design variants with real users on a measurable goal

📖 GFG: Usability Principles

5.5 Aspect-Oriented Software Engineering (AOSE)

📘 Definition

AOSE / AOP improves modularity by explicitly separating **cross-cutting concerns** — logging, security, transaction management, error handling — that would otherwise be scattered and tangled across unrelated modules.

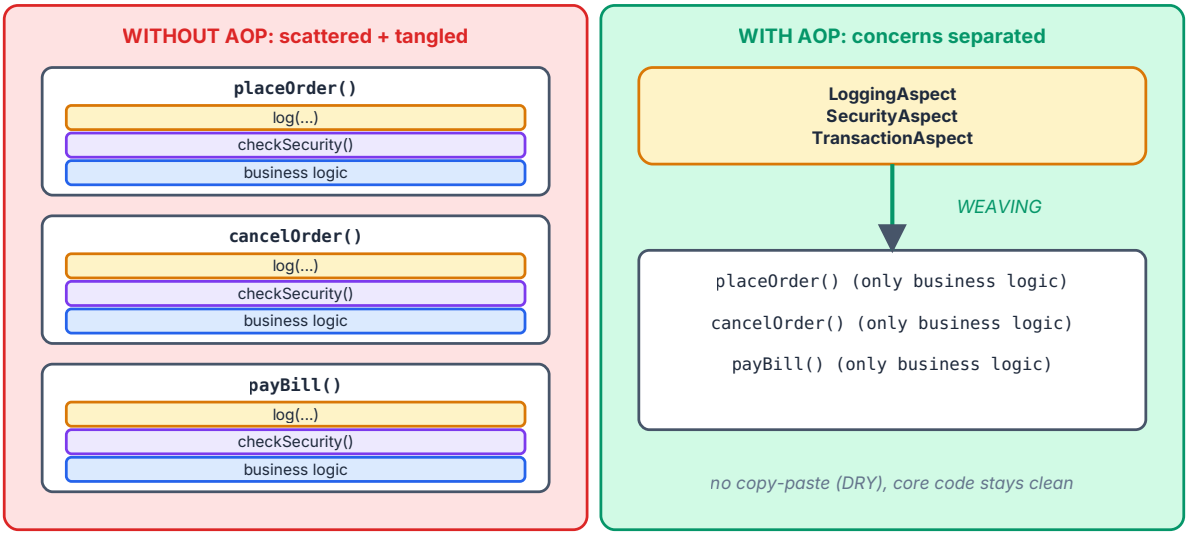


Fig. 50 — Without AOP: concerns scattered and tangled. With AOP: one aspect, woven in

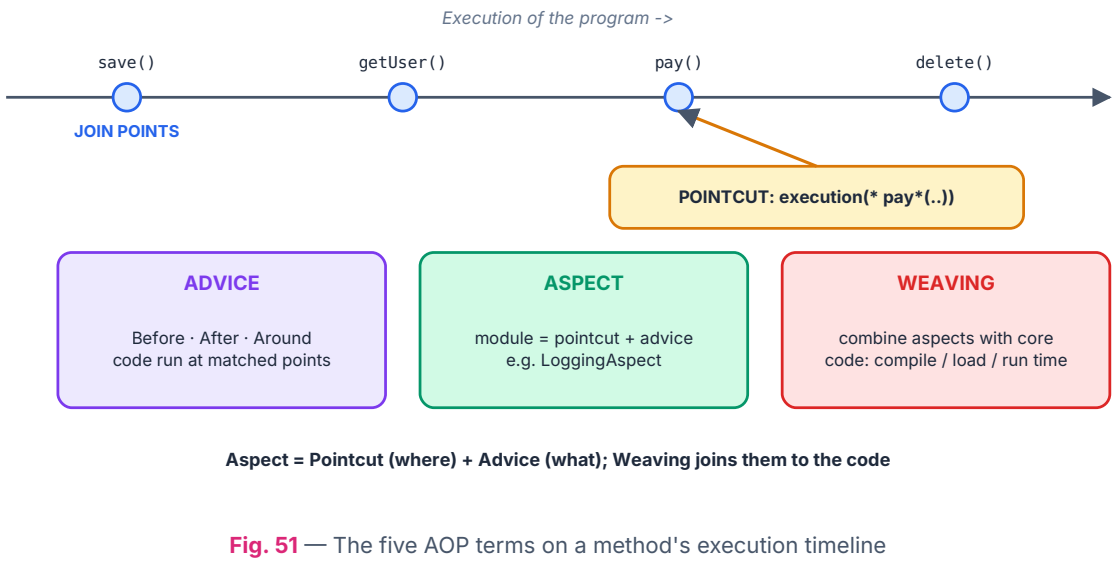


Fig. 51 — The five AOP terms on a method's execution timeline

Term	Meaning
Aspect	Module encapsulating a cross-cutting concern (e.g. LoggingAspect)
Join point	A point in execution (e.g. a method call) where an aspect can apply
Pointcut	Expression selecting the join points where an aspect applies
Advice	The action run at a matched join point — Before, After, Around
Weaving	Combining aspects with core code at compile-time, load-time or run-time

Without AOP, logging or security checks must be duplicated in every method — violating **DRY** and hiding the business logic. AOP weaves them in separately.

🧠 **Yaad rakho**

A-J-P-A-W: Aspect, Join point, Pointcut, Advice, Weaving. Pointcut = **kahan**, Advice = **kya**.

📖 GFG: AOP in Spring

5.6 Cleanroom Software Engineering

📘 **Definition**

Cleanroom is a rigorous, formal-methods-based approach whose central goal is **defect prevention, not defect removal** — certifiably reliable software with (ideally) zero defects.

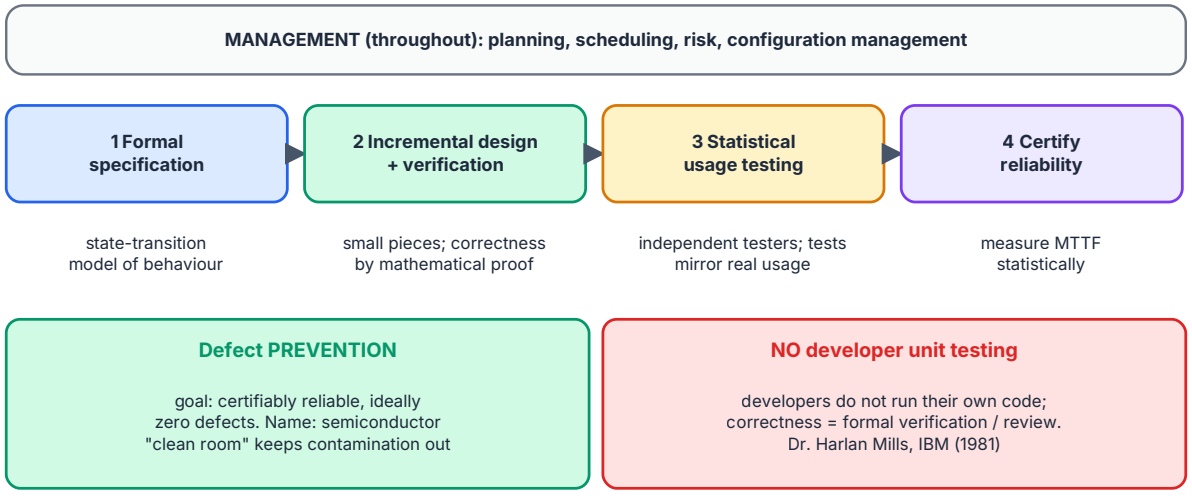


Fig. 52 — Four Cleanroom processes

Developed by **Dr. Harlan Mills at IBM** (released 1981, wider adoption after 1987). The name comes from the semiconductor "clean room": keep contamination (defects) out, rather than cleaning up afterward.

1. **Management** — throughout: planning, scheduling, resources, risk, configuration management.
2. **Specification** — formal requirements, often a state-transition model.
3. **Design & verification** — incremental development with formal design, correctness verified by **mathematical proof**, not ad hoc testing.
4. **Statistical quality control / testing** — developers do **not** unit-test their own code; independent testers use **statistical usage testing** driven by expected real-world use to certify reliability.

> ⭐ **Exam tip**
>
> Key phrase: **"no unit testing by developers; correctness by formal verification; certification by statistical usage testing."**

📖 GFG: Cleanroom Software Engineering

5.7 Crowdsourcing in Software Engineering

> 📘 **Definition**
>
> **Crowdsourcing** distributes software tasks (coding, testing, design, requirements, bug-finding) to a large, often global pool of external contributors via open calls, instead of relying only on an in-house team.

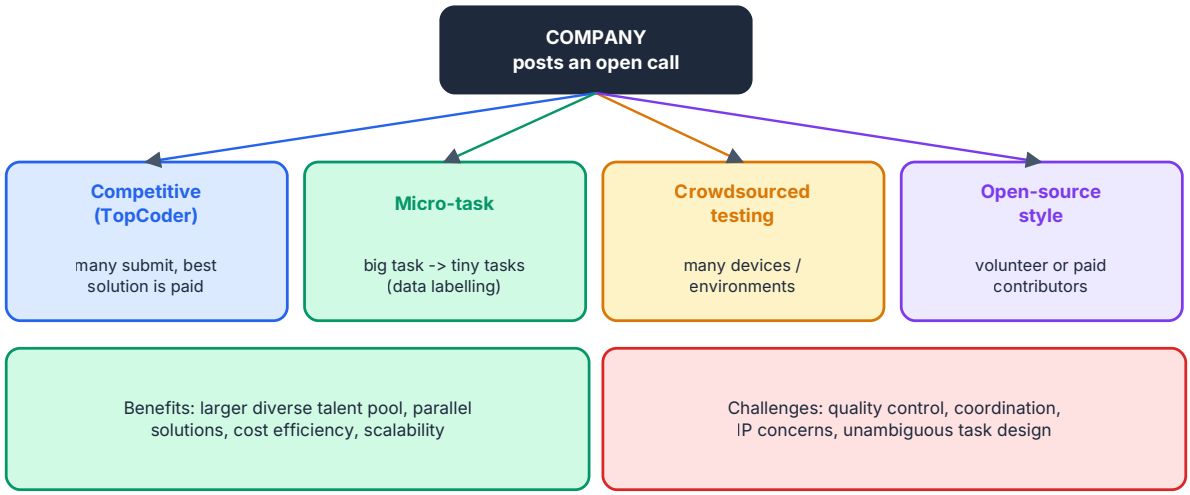


Fig. 53 — Four crowdsourcing models, with benefits and challenges

Model	Idea
Competitive platforms (e.g. TopCoder)	Many contributors compete; usually only the winning solution is paid
Micro-task	Large task split into small independent tasks — data labelling, multi-device testing
Crowdsourced testing	Diverse testers on their own devices find bugs an in-house QA team misses
Open-source-style	Volunteer or paid contributors collaboratively build and maintain software

Benefits	Challenges
Larger, more diverse talent pool; parallel solutions; cost efficiency; scalability for parallel tasks (data annotation)	Quality control; coordination overhead; IP concerns; writing unambiguous tasks for a distributed workforce

5.8 Artificial Intelligence & Machine Learning in SDLC

> 📘 **Definition**
>
> AI / ML is increasingly integrated into **every SDLC phase**, **augmenting — not replacing** — human developers.

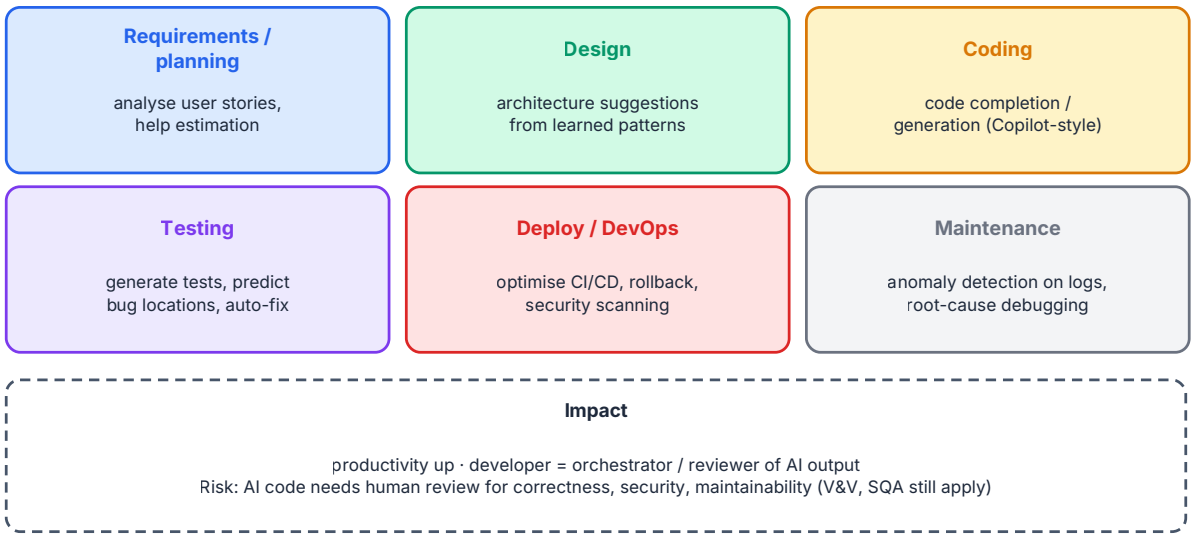


Fig. 54 — What AI / ML contributes at each SDLC phase

SDLC phase	AI / ML contribution
Requirements / planning	Analyse requirements, user stories, repositories for planning and estimation
Design	Architecture suggestions from patterns learned in large codebases
Coding	Code completion / generation assistants (Copilot-style); less boilerplate
Testing	Generate unit / integration tests, fuzz edge cases, predict bug locations from defect history, auto-fix some failing tests
Deployment / DevOps	Optimise CI/CD, suggest safe rollbacks, scan for vulnerabilities in real time
Maintenance	Anomaly detection on logs / metrics to predict failures; AI-assisted debugging and root-cause analysis

Overall impacts

- **Productivity gains** — shorter time-to-market, more developer output.
- **Role shift** — from writing every line to **orchestrating / reviewing** AI output; more focus on strategy, architecture, trade-offs.
- **New risks** — AI code still needs human review for correctness, security and maintainability; over-reliance without verification (Unit IV's V&V / SQA) can hurt quality and reliability.

> ⚠️ **Careful**
>
> AI-generated code is **not** automatically verified. Verification and validation still apply — treat it like code from a new junior developer.

📖 GFG: AI in Software Development

❓ Likely exam questions — Unit V

- Define HCI and usability (ISO 9241); usability attributes; Eight Golden Rules.
- Evolution of user interfaces: CLI → GUI → NUI → conversational.
- Gulf of execution and gulf of evaluation.
- Component-based GUI development and its benefits.
- Usability engineering process; evaluation methods.
- Explain AOSE: aspect, join point, pointcut, advice, weaving.
- Cleanroom software engineering and its four processes.
- Crowdsourcing models, benefits and challenges.
- Role of AI / ML in each SDLC phase; risks.

Quick comparison sheet

One-glance tables for the last day before the exam. Each row is a favourite "differentiate between" question.

Reuse, architecture and evolution

Topic	Remember
Vertical / horizontal reuse	Same domain / across domains
Black-box / white-box	Interface only / source visible and modifiable
Opportunistic / systematic	Ad hoc / planned from the start
CBSE for / with reuse	Build reusable components / assemble existing ones
CBSE process	Qualify → Adapt → Compose → Update
Domain / application engineering	Build family assets / build products from them
Application / component system	Full end-user product / reusable unit with interface
Facade	One simple interface hiding internals; reduces coupling
Layers	Presentation → Business → Domain/Service → Data
Maintenance types	Corrective, Adaptive, Perfective, Preventive
Reengineering pipeline	Inventory → Document restructuring → Reverse → Code/Data restructuring → Forward
Reverse / forward engineering	Code → design (up) / design → code (down)
Restructuring	Same abstraction level, same behaviour
Software aging	Code rot / runtime aging (fix: rejuvenation)

Dependability, quality and CASE

Topic	Remember
Dependability	Availability + Reliability + Safety + Security + Maintainability
Reliable vs safe	Rarely fails vs fails harmlessly — not the same
Availability	MTBF / (MTBF + MTTR)
Reliability approaches	Fault avoidance, tolerance, detection & removal
Jelinski-Moranda	Failure rate ∝ remaining faults, drops after each fix
McCall factors	Operation 5 · Revision 3 · Transition 3
QA / QC	Process, prevent / product, detect
Verification / validation	Built right (static) / right product (dynamic)
SQA	Preventive umbrella: standards, audits, defect tracking, CMMI, training
CASE generations	Isolated tools / I-CASE shared repository / modern round-trip
CASE architecture	User interface + Tool set + OMS + Repository

Usability and advanced paradigms

Topic	Remember
Usability (ISO 9241)	Learnability, Efficiency, Memorability, Errors, Satisfaction
UI evolution	CLI → menu → GUI (WIMP) → NUI → conversational
Usability evaluation	Usability test, heuristic, cognitive walkthrough, A/B
AOP	Aspect, Join point, Pointcut, Advice, Weaving
Cleanroom	Prevention + formal proof + statistical usage testing; no developer unit testing
Crowdsourcing	Competitive, micro-task, crowd testing, open-source style
AI in SDLC	Assists every phase; human review still required

Exam-day revision checklist

If you can **write or draw** each of these from memory, you cover the highest-weightage recurring questions.

- [ ] Definition and levels of software reuse
- [ ] Vertical vs horizontal; black-box vs white-box reuse
- [ ] Six reuse approaches
- [ ] Benefits and barriers (NIH syndrome)
- [ ] Success and failure factors; maturity ladder
- [ ] CBSE process: Qualify → Adapt → Compose → Update
- [ ] Domain engineering vs application engineering
- [ ] Application system vs component system
- [ ] Application family and why 60–90 % reuse
- [ ] Steps to build an application from components
- [ ] Variability mechanisms (5)
- [ ] Facade pattern diagram and benefits
- [ ] Layered architecture (4 layers)
- [ ] Lehman's laws
- [ ] Four maintenance types; three models; metrics
- [ ] Reengineering pipeline diagram
- [ ] Program comprehension: static vs dynamic
- [ ] Reverse engineering steps; restructuring types
- [ ] Forward engineering; re-documentation
- [ ] Software aging and rejuvenation
- [ ] Dependability attributes
- [ ] Reliable vs safe; safety techniques
- [ ] Availability = MTBF / (MTBF + MTTR) numerical
- [ ] Reliability metrics; Jelinski-Moranda model
- [ ] McCall quality factors; QA vs QC
- [ ] Verification vs validation; V-model
- [ ] SQA activities
- [ ] CASE scope, SDLC support, generations, architecture
- [ ] Usability attributes; Shneiderman's rules
- [ ] UI evolution; gulfs of execution / evaluation
- [ ] Usability process and evaluation methods
- [ ] AOP terms; Cleanroom processes; crowdsourcing; AI in SDLC

Recommended resources

Resource	Use it for
GeeksforGeeks — Software Engineering	Topic-wise articles
Sommerville — *Software Engineering*	Reuse, CBSE, dependability, evolution (standard textbook)
Pressman — *Software Engineering: A Practitioner's Approach*	Reengineering, quality, SQA, Cleanroom
Nielsen Norman — 10 usability heuristics	Heuristic evaluation
Spring Framework — AOP	Aspect / advice / pointcut in practice

Diagrams in this book are original drawings made for these notes; use them as a model for the figures you draw in the exam.